

Program Repair and Automated Debugging with Code LLMs: A Survey

Program repair and automated debugging are being reshaped by code-oriented large language models. Where earlier automated program repair systems relied mainly on templates, search, constraints, or hand-engineered mutation operators, code LLMs can synthesize patches, explain failures, localize faults, generate tests, and interact with repositories through natural-language and execution feedback. This survey synthesizes an arXiv-derived corpus of 300 papers collected during this run and organizes the literature around patch generation, debugging and fault localization, test-based validation, repository-level coding agents, program analysis guidance, vulnerability repair, training signals, benchmarks, and reliability. The survey argues that LLM-based repair is best understood as a closed-loop software engineering system: models propose edits, tools execute tests and analyses, feedback revises hypotheses, and benchmarks measure whether patches are correct beyond superficial plausibility. We close with open problems in overfitting to tests, repository context, secure repair, human oversight, and reproducible evaluation.

Keywords:

large language models; program repair; automated debugging; code LLMs; software engineering; fault localization; testing

1 Introduction

Automated program repair and debugging have long pursued a practical goal: reduce the time between a software failure and a correct, maintainable fix. Traditional systems search over edits, mine templates, solve constraints, or use tests to distinguish plausible patches from incorrect ones. Code-oriented large language models change this landscape because they bring broad code priors, natural-language issue understanding, multi-file editing ability, and interactive use of compilers, tests, static analyzers, and repository tools [21, 135, 146, 168, 171, 186, 199, 286]. The repair problem is therefore no longer only a mutation search problem; it is also a language, reasoning, retrieval, and feedback-control problem.

The appeal of code LLMs is clear. A model can read an issue report, inspect source files, hypothesize the failing component, generate a candidate patch, run tests, interpret stack traces, revise the edit, and explain the result to a developer. The risk is equally clear: a patch may satisfy visible tests while preserv-

ing the underlying bug, introducing a security flaw, or reducing maintainability. LLM-based repair inherits the classical automated program repair problem of *patch correctness*, but it adds new concerns about hallucinated APIs, incomplete repository context, brittle prompting, and benchmark contamination [21, 135, 146, 168, 171, 186, 199, 286].

This survey reviews program repair and automated debugging with code LLMs as an integrated software engineering workflow. We focus on how papers formulate failures, represent repository context, localize defects, synthesize edits, validate patches, use execution feedback, train or adapt code models, and evaluate systems on repair benchmarks. The central claim is that reliable repair requires a loop: hypothesis, edit, execution, diagnosis, revision, and human-facing explanation.

2 Background

2.1 From Automated Program Repair to LLM-Based Repair

Automated program repair (APR) traditionally includes generate-and-validate systems, semantics-based repair, template mining, search-based mutation, and learning-based patch generation. These approaches depend heavily on tests, specifications, or localized search spaces. Code LLMs expand the search space because they can synthesize longer edits and reason over natural-language bug reports, but they also make validation harder. A fluent patch is not necessarily a correct patch, and a model’s explanation may not faithfully describe the semantic change [21, 135, 146, 168, 171, 186, 199, 286].

2.2 Debugging as Hypothesis Refinement

Debugging is the process of explaining why observed behavior differs from intended behavior. LLM-based debugging systems typically combine source-code understanding, stack traces, compiler errors, failing tests, logs, and natural-language issue descriptions. Fault localization narrows the search region; diagnosis predicts the failure mechanism; repair modifies code; and validation checks whether the hypothesis was right. Modern code agents blur these stages by interleaving them in one loop [21, 135, 137, 146, 153, 186, 199, 286].

2.3 Execution Feedback and the Role of Tests

Tests are the main grounding mechanism for program repair. They provide an executable oracle, but often an incomplete one. LLM-based repair systems use tests to filter patches, generate additional tests, reproduce bugs, and guide self-correction after failures. The classical risk of test-suite overfitting remains central: a patch can pass available tests while failing hidden behavior, violating specifications, or changing public APIs [21, 54, 63, 127, 135, 153, 168, 186].

3 Methods

3.1 Search Protocol and Corpus Construction

The corpus for this survey was generated during this run from the arXiv structured API. Queries combined the task topic with program repair, automated debugging, code LLMs, patch generation,

bug fixing, fault localization, unit tests, execution feedback, SWE-bench, repository-level agents, static analysis, program analysis, compiler and runtime errors, code review, vulnerability repair, software testing, patch correctness, benchmarks, and training terms. Records were retained when title or abstract evidence connected LLMs or language models with repair, debugging, testing, repository-level software engineering, program analysis, code security, training, evaluation, or reliability. Duplicates were removed by arXiv identifier, and the final bibliography cites 300 unique scholarly preprints.

3.2 Taxonomy

We organize the literature along seven axes. The first is *failure signal*: bug report, failing test, exception, compiler error, static-analysis warning, vulnerability report, or developer feedback. The second is *context scope*: function, file, package, repository, dependency graph, or issue history. The third is *localization*: retrieval, static analysis, stack trace reasoning, spectrum-based ranking, model attention, or agent exploration. The fourth is *repair action*: single-line edit, multi-hunk patch, refactoring, configuration change, dependency update, or test update. The fifth is *feedback*: compilation, unit tests, generated tests, symbolic checks, runtime traces, review comments, or human approval. The sixth is *learning*: prompting, supervised fine-tuning, reinforcement learning, preference optimization, self-repair, or distillation. The seventh is *evaluation*: plausibility, hidden-test correctness, maintainability, security, cost, and developer trust.

3.3 Patch Generation

Patch generation is the most visible use of code LLMs in repair. Systems may prompt a model with a bug report and a code region, retrieve related files, generate candidate patches, and rank them using tests or learned scores. Compared with template-based APR, LLMs can produce semantically rich edits and operate over unfamiliar APIs, but they can also invent nonexistent functions or make broad changes that are difficult to review. Strong systems constrain the edit scope, preserve style, and connect each change to an explicit failure hypothesis [21, 135, 146, 168, 171, 186, 199, 286].

3.4 Fault Localization and Automated Debugging

Before repair, a system must decide where and why the program is failing. LLM-based localization uses

stack traces, compiler messages, natural-language bug reports, call graphs, execution traces, and retrieved code snippets. Debugging agents can ask targeted questions of the repository: which tests fail, which files changed recently, where is this symbol defined, and what invariant is violated? The best results arise when language-model reasoning is combined with concrete signals from program analysis and execution [21, 135, 137, 146, 153, 186, 199, 286].

3.5 Test Generation and Validation

LLMs can help on both sides of testing. They can write reproduction tests from bug reports, create regression tests for candidate patches, and generate adversarial tests that reveal overfitting. They can also interpret failures and decide whether a patch should be revised. However, generated tests may encode the model’s mistaken assumptions, so validation should combine model-generated tests with existing suites, hidden tests, coverage analysis, and, when possible, formal or domain-specific specifications [21, 54, 63, 127, 135, 153, 168, 186].

3.6 Repository-Level Coding Agents

Repository-level repair requires more than editing a local snippet. The system must navigate project structure, build commands, dependencies, style conventions, APIs, issue reports, and multi-file interactions. SWE-bench-style evaluations have made this setting influential because they measure whether an agent can resolve real repository issues rather than toy examples. Repository agents use retrieval, planning, shell commands, search tools, patch application, and iterative test execution; their failures often stem from missing context or poor recovery after an execution error [21, 135, 146, 168, 171, 186, 199, 286].

3.7 Program Analysis and Specification Guidance

Program analysis can make LLM repair more grounded. Static analysis, type checking, data-flow analysis, symbolic execution, contracts, and compiler diagnostics provide structured evidence about what code can do and why it may fail. These tools reduce the burden on the language model to infer semantics from text alone. A productive direction is a division of labor: models generate hypotheses and candidate edits, while analyzers supply constraints, counterexamples, and localized evidence [35, 37, 84, 121, 178, 279, 287, 300].

3.8 Security Bugs and Vulnerability Repair

Repair for security vulnerabilities is especially high stakes. A model may silence a warning without removing the exploit path, introduce a new vulnerability, or leak sensitive implementation details in an explanation. Security repair also requires reasoning about attacker behavior, input validation, privileges, cryptography, concurrency, and deployment configuration. LLM systems should therefore pair patch generation with vulnerability-specific tests, static analyzers, review policies, and conservative human approval [54, 87, 121, 133, 153, 209, 279, 286].

3.9 Training and Feedback for Code LLMs

Prompting alone can produce useful repairs, but training and adaptation can teach models to use feedback more reliably. Supervised data can pair bugs with patches; preference data can rank maintainable fixes; reinforcement learning can reward hidden-test success; and self-debugging traces can teach revision after compiler or test failures. The challenge is credit assignment: a failed repair may reflect wrong localization, missing context, a bad edit, weak tests, or an unreliable verifier [4, 54, 87, 127, 133, 137, 168, 211].

3.10 Benchmarks and Evaluation

Evaluation should distinguish plausible patches from correct repairs. Passing visible tests is a weak signal when tests are incomplete or when benchmark instances are seen during training. Strong evaluations use held-out repositories, hidden tests, issue-level task definitions, build reproducibility, cost accounting, patch review, and security or maintainability checks. Benchmarks also need trace logs so researchers can identify whether failures come from search, localization, editing, execution, or stopping decisions [21, 135, 137, 153, 168, 171, 186, 286].

3.11 Representative Literature Map

The following map cites the full retained corpus by theme, making the survey coverage explicit.

Automated program repair and patch generation
The automated program repair and patch generation cluster contains work that treats LLMs as controllers of typed resources, stateful environments, or evaluation tasks [21, 127, 133, 135, 137, 146, 153, 186, 199]. A second strand in this cluster studies how such systems represent intermediate state, recover from

failed calls, and compare tool-mediated behavior across datasets [4, 15, 54, 87, 119, 121, 209, 211, 279]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [37, 76, 84, 103, 118, 175, 190, 237, 300]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [7, 77, 89, 176, 193, 218, 225, 233, 282]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [18, 19, 41, 55, 62, 198, 202, 234, 244]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [104, 116, 134, 150, 180, 245, 249, 274, 294]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [6, 72, 102, 142, 144, 152, 215, 246, 254]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [5, 13, 50, 136, 151, 165, 243, 258, 260]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [31, 43, 97, 114, 141, 148, 238, 242, 283]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [32, 33, 46, 91, 122, 187, 259, 263, 292]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [75, 78, 82, 94, 111, 166, 241, 256, 262]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [125, 191, 201, 203, 205, 208, 235, 284, 293]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [47, 85, 101, 143, 155, 163, 184, 289, 297]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [17, 29, 38, 66, 110, 157, 177, 185, 273]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [34, 42, 48, 57, 113, 162, 172, 222, 250]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [45, 53, 132, 154, 160, 188, 219, 220, 280]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [11, 24, 83, 90, 129, 179, 217, 236, 275]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [79, 117, 164, 173, 196].

Debugging, fault localization, and diagnosis The debugging, fault localization, and diagnosis cluster contains work that treats LLMs as controllers of typed resources, stateful environments, or evaluation tasks [23, 52, 81, 195, 232, 240, 257, 269, 276]. A second strand in this cluster studies how such systems represent intermediate state, recover from failed calls, and compare tool-mediated behavior across datasets [16, 65, 120, 131, 161, 181, 214, 270, 272]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [271].

Test generation, validation, and execution feedback The test generation, validation, and execution feedback cluster contains work that treats LLMs as controllers of typed resources, stateful environments, or evaluation tasks [35, 36, 58, 69, 109, 130, 170, 178, 183]. A second strand in this cluster studies how such systems represent intermediate state, recover from failed calls, and compare tool-mediated behavior across datasets [8, 39, 159, 169, 212, 224, 251, 252, 264]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [1, 14, 59, 60, 96, 221, 229, 290, 296]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [2, 74, 106, 189, 197, 210, 223, 266, 267]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [67, 80, 99, 105, 139, 156, 265, 278, 291]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [44, 73, 112, 128, 192, 206, 207, 231, 268]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [28, 115, 138, 285].

Repository-level coding agents and swe-bench The repository-level coding agents and swe-bench cluster contains work that treats LLMs as controllers of typed resources, stateful environments, or evaluation tasks [63, 123, 140, 167, 168, 171, 228, 286, 287]. A second strand in this cluster studies how such systems represent intermediate state, recover from failed calls, and compare tool-mediated behavior across datasets [64, 70, 124, 158, 213, 216, 261, 277, 295]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [9, 20, 71, 88, 95, 149, 247, 248, 281].

Program analysis, specifications, and tool guidance The program analysis, specifications, and tool guidance cluster contains work that treats LLMs as controllers of typed resources, stateful environments, or evaluation tasks [98, 182].

Code review, vulnerability repair, and security bugs The code review, vulnerability repair, and security bugs cluster contains work that treats LLMs as controllers of typed resources, stateful environments, or evaluation tasks [49, 126, 227, 288].

Training, adaptation, and feedback for code llms The training, adaptation, and feedback for code llms cluster contains work that treats LLMs as controllers of typed resources, stateful environments, or evaluation tasks [92, 93].

Benchmarks, evaluation, and reliability The benchmarks, evaluation, and reliability cluster contains work that treats LLMs as controllers of typed resources, stateful environments, or evaluation tasks [22, 26, 51, 68, 145, 194, 200, 255, 299]. A second strand in this cluster studies how such systems represent intermediate state, recover from failed calls, and compare tool-mediated behavior across datasets [27, 30, 61, 100, 108, 147, 204, 226, 253]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [3, 10, 12, 25, 56, 86, 174, 239, 298]. Additional retained studies broaden the cluster with variants of interfaces, domains, training signals, and deployment assumptions [40, 107, 230].

4 Challenges and Prospects

4.1 Patch Correctness Beyond Plausibility

The central challenge remains correctness. A patch can compile and pass available tests while encoding the wrong semantics. LLMs make this problem more subtle because they produce natural and idiomatic code even when the underlying hypothesis is false. Future repair systems need stronger oracles, generated counterexamples, specification mining, and review interfaces that expose why a patch should be trusted.

4.2 Repository Context and Long-Horizon Control

Real bugs live in repositories with build systems, dependencies, style conventions, and historical design

decisions. Agents must decide what context to retrieve, when to run expensive commands, and how to recover when the environment fails. Long-horizon control, memory, and tool-use research are therefore directly relevant to repair.

4.3 Feedback Loops and Test Overfitting

Execution feedback is powerful but can encourage overfitting. If an agent repeatedly edits until visible tests pass, it may learn to satisfy the test suite rather than the specification. Better systems will generate independent tests, use coverage and mutation analysis, compare alternative patches, and preserve failed traces for audit.

4.4 Security, Maintainability, and Human Oversight

Repair systems should optimize for maintainable and secure fixes, not only benchmark success. Human developers need compact explanations, patch rationales, risk flags, and diffs that fit review practice. High-impact or security-critical repairs should remain approval-gated until automated evidence is strong enough to justify trust [54, 87, 121, 133, 153, 209, 279, 286].

4.5 Reproducibility and Benchmark Hygiene

Code repair evaluation is vulnerable to environment drift, dependency failures, and data contamination. The field needs versioned repositories, stable build images, transparent logs, and benchmark splits that reduce leakage into model training data. Without such hygiene, apparent progress may reflect memorization or tooling advantages rather than better debugging ability.

5 Conclusion

Program repair and automated debugging with code LLMs are moving from isolated patch synthesis toward closed-loop software engineering agents. This survey reviewed patch generation, localization, testing, repository-level repair, program analysis guidance, vulnerability repair, training, benchmarks, and reliability. The central lesson is that code LLMs are most valuable when grounded by executable feedback and constrained by software engineering evidence. Future systems should combine model priors with tests, analysis, specifications, provenance, and

human oversight so that repair becomes not merely plausible but trustworthy.

References

- [1] Azat Abdullin, Pouria Derakhshanfar, and Annibale Panichella. Test Wars: A Comparative Study of SBST, Symbolic Execution, and LLM-Based Approaches to Unit Test Generation. *arXiv preprint arXiv:2501.10200*, 2025. URL <https://arxiv.org/abs/2501.10200>.
- [2] Anisha Agarwal, Aaron Chan, Shubham Chandel, Jinu Jang, Shaun Miller, Roshanak Zilouchian Moghaddam, Yevhen Mohylevskyy, Neel Sundaresan, and Michele Tufano. Copilot Evaluation Harness: Evaluating LLM-Guided Software Programming. *arXiv preprint arXiv:2402.14261*, 2024. URL <https://arxiv.org/abs/2402.14261>.
- [3] Wasi Uddin Ahmad, Aleksander Ficek, Mehrzad Samadi, Jocelyn Huang, Vahid Noroozi, Somshubra Majumdar, and Boris Ginsburg. OpenCodeInstruct: A Large-scale Instruction Tuning Dataset for Code LLMs. *arXiv preprint arXiv:2504.04030*, 2025. URL <https://arxiv.org/abs/2504.04030>.
- [4] Nikta Akbarpour, Mahdiah Sadat Benis, Fatemeh Hendijani Fard, Ali Ouni, and Mohamed Aymen Saied. Collaborative Agents for Automated Program Repair in Ruby. *arXiv preprint arXiv:2511.03925*, 2025. URL <https://arxiv.org/abs/2511.03925>.
- [5] Amir Al-Maamari. Why LLMs Fail: A Failure Analysis and Partial Success Measurement for Automated Security Patch Generation. *arXiv preprint arXiv:2603.10072*, 2026. URL <https://arxiv.org/abs/2603.10072>.
- [6] Alex, Liu, Vivian, and Chi. From Defects to Demands: A Unified, Iterative, and Heuristically Guided LLM-Based Framework for Automated Software Repair and Requirement Realization. *arXiv preprint arXiv:2412.05098*, 2024. URL <https://arxiv.org/abs/2412.05098>.
- [7] Avinash Anand, Akshit Gupta, Nishchay Yadav, and Shaurya Bajaj. A Comprehensive Survey of AI-Driven Advancements and Techniques in Automated Program Repair and Code Generation. *arXiv preprint arXiv:2411.07586*, 2024. URL <https://arxiv.org/abs/2411.07586>.
- [8] Gabor Antal, Denes Ban, Martin Isztin, Rudolf Ferenc, and Peter Hegedus. Leveraging GPT-4 for Vulnerability-Witnessing Unit Test Generation. *arXiv preprint arXiv:2506.11559*, 2025. URL <https://arxiv.org/abs/2506.11559>.
- [9] Moumita Asad, Rafed Muhammad Yasir, and Sam Malek. Towards Explorative IRBL: Combining Semantic Retrieval with LLM-driven Iterative Code Exploration. *arXiv preprint arXiv:2508.00253*, 2025. URL <https://arxiv.org/abs/2508.00253>.
- [10] Maider Azanza, Beatriz Perez Lamancha, and Eneko Pizarro. Tracking the Moving Target: A Framework for Continuous Evaluation of LLM Test Generation in Industry. *arXiv preprint arXiv:2504.18985*, 2025. URL <https://arxiv.org/abs/2504.18985>.
- [11] Razan Baltaji, Saurabh Pujar, Louis Mandel, Martin Hirzel, Luca Buratti, and Lav Varshney. Cross-lingual Transfer in Programming Languages: An Extensive Empirical Study. *arXiv preprint arXiv:2310.16937*, 2023. URL <https://arxiv.org/abs/2310.16937>.
- [12] Vekil Bekmyradov, Noah C. Putz, and Thomas Bartz-Beielstein. LLMs taking shortcuts in test generation: A study with SAP HANA and LevelDB. *arXiv preprint arXiv:2604.14437*, 2026. URL <https://arxiv.org/abs/2604.14437>.
- [13] Berkay Berabi, Alexey Gronskiy, Veselin Raychev, Gishor Sivanrupan, Victor Chibotaru, and Martin Vechev. DeepCode AI Fix: Fixing Security Vulnerabilities with Large Language Models. *arXiv preprint arXiv:2402.13291*, 2024. URL <https://arxiv.org/abs/2402.13291>.
- [14] Shreya Bhatia, Tarushi Gandhi, Dhruv Kumar, and Pankaj Jalote. Unit Test Generation using Generative AI : A Comparative Performance Analysis of Autogeneration Tools. *arXiv preprint arXiv:2312.10622*, 2023. URL <https://arxiv.org/abs/2312.10622>.
- [15] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. *arXiv preprint arXiv:2403.17134*, 2024. URL <https://arxiv.org/abs/2403.17134>.
- [16] Ricardo Brancas, Vasco Manquinho, and Ruben Martins. Combining Logic with Large Language Models for Automatic Debugging and Repair of ASP Programs. *arXiv preprint arXiv:2410.20962*, 2024. URL <https://arxiv.org/abs/2410.20962>.
- [17] Xuemeng Cai and Lingxiao Jiang. Adapting Knowledge Prompt Tuning for Enhanced Automated Program Repair. *arXiv preprint arXiv:2504.01523*, 2025. URL <https://arxiv.org/abs/2504.01523>.
- [18] Viola Campos, Ridwan Shariffdeen, Adrian Ulges, and Yannic Noller. Exploring Generalizable Automated Program Repair with Large Language Models. *arXiv preprint arXiv:2506.03283*, 2025. URL <https://arxiv.org/abs/2506.03283>.

- [19] Xin Cao and Nan Yu. SIADAFIX: issue description response for adaptive program repair. *arXiv preprint arXiv:2510.16059*, 2025. URL <https://arxiv.org/abs/2510.16059>.
- [20] Genevieve Caumartin and Glaucia Melo. Reformulate, Retrieve, Localize: Agents for Repository-Level Bug Localization. *arXiv preprint arXiv:2512.07022*, 2025. URL <https://arxiv.org/abs/2512.07022>.
- [21] Ira Ceka, Hailie Mitchell, Saurabh Pujar, Luca Burratti, Shyam Ramji, Junfeng Yang, Gail Kaiser, and Baishakhi Ray. Understanding Automated Program Repair Agents Through the Lens of Traceability: An Empirical Study. *arXiv preprint arXiv:2506.08311*, 2025. URL <https://arxiv.org/abs/2506.08311>.
- [22] Partha Chakraborty, Mahmoud Alfadel, and Meiyappan Nagappan. BLAZE: Cross-Language and Cross-Project Bug Localization via Dynamic Chunking and Hard Example Learning. *arXiv preprint arXiv:2407.17631*, 2024. URL <https://arxiv.org/abs/2407.17631>.
- [23] Jianming Chang, Xin Zhou, Lulu Wang, David Lo, and Bixin Li. Bridging Bug Localization and Issue Fixing: A Hierarchical Localization Framework Leveraging Large Language Models. *arXiv preprint arXiv:2502.15292*, 2025. URL <https://arxiv.org/abs/2502.15292>.
- [24] Aolin Chen, Haojun Wu, Qi Xin, Steven P. Reiss, and Jifeng Xuan. Studying and Understanding the Effectiveness and Failures of Conversational LLM-Based Repair. *arXiv preprint arXiv:2503.15050*, 2025. URL <https://arxiv.org/abs/2503.15050>.
- [25] Jiaao Chen, Jingyuan Qi, Mingye Gao, Wei-Chen Wang, Hanrui Wang, and Di Jin. EigenData: A Self-Evolving Multi-Agent Platform for Function-Calling Data Synthesis, Auditing, and Repair. *arXiv preprint arXiv:2603.05553*, 2026. URL <https://arxiv.org/abs/2603.05553>.
- [26] Jialong Chen, Xander Xu, Hu Wei, Chuan Chen, and Bing Zhao. SWE-CI: Evaluating Agent Capabilities in Maintaining Codebases via Continuous Integration. *arXiv preprint arXiv:2603.03823*, 2026. URL <https://arxiv.org/abs/2603.03823>.
- [27] Xinyun Chen, Maxwell Lin, Nathanael Scharli, and Denny Zhou. Teaching Large Language Models to Self-Debug. *arXiv preprint arXiv:2304.05128*, 2023. URL <https://arxiv.org/abs/2304.05128>.
- [28] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. ChatUnitTest: A Framework for LLM-Based Test Generation. *arXiv preprint arXiv:2305.04764*, 2023. URL <https://arxiv.org/abs/2305.04764>.
- [29] Yuxiao Chen, Jingzheng Wu, Xiang Ling, Changjiang Li, Zhiqing Rui, Tianyue Luo, and Yanjun Wu. When Large Language Models Confront Repository-Level Automatic Program Repair: How Well They Done? *arXiv preprint arXiv:2403.00448*, 2024. URL <https://arxiv.org/abs/2403.00448>.
- [30] Zaoyu Chen, Haoran Qin, Nuo Chen, Xiangyu Zhao, Lei Xue, Xiapu Luo, and Xiaoming Wu. Towards Automated Smart Contract Generation: Evaluation, Benchmarking, and Retrieval-Augmented Repair. *arXiv preprint arXiv:2503.01098*, 2025. URL <https://arxiv.org/abs/2503.01098>.
- [31] Runxiang Cheng, Michele Tufano, Jurgen Cito, Jose Cambronero, Pat Rondon, Renyao Wei, Aaron Sun, and Satish Chandra. Agentic Bug Reproduction for Effective Automated Program Repair at Google. *arXiv preprint arXiv:2502.01821*, 2025. URL <https://arxiv.org/abs/2502.01821>.
- [32] Xiao Cheng, Zhihao Guo, Huan Huo, and Yulei Sui. Tracing Errors, Constructing Fixes: Repository-Level Memory Error Repair via Typestate-Guided Context Retrieval. *arXiv preprint arXiv:2506.18394*, 2025. URL <https://arxiv.org/abs/2506.18394>.
- [33] Yu Cheng, Zhongxin Liu, Zhenchang Xing, Chao Ni, Qing Huang, and Xiaoxue Ren. PracRepair: LLM-Empowered Automated Program Repair Inspired by Human-Like Debugging Practices. *arXiv preprint arXiv:2606.17612*, 2026. URL <https://arxiv.org/abs/2606.17612>.
- [34] Mohd Sameen Chishty, Damilare Peter Oyinloye, and Jingyue Li. AgentReputation: A Decentralized Agentic AI Reputation Framework. *arXiv preprint arXiv:2605.00073*, 2026. URL <https://arxiv.org/abs/2605.00073>.
- [35] Jaid Monwar Chowdhury, Chi-An Fu, and Reyhaneh Jabbarvand. SPARC: Scenario Planning and Reasoning for Automated C Unit Test Generation. *arXiv preprint arXiv:2602.16671*, 2026. URL <https://arxiv.org/abs/2602.16671>.
- [36] Bei Chu, Yang Feng, Kui Liu, Zhaoqiang Guo, Yichi Zhang, Hange Shi, Zifan Nan, and Baowen Xu. Large Language Models for Unit Test Generation: Achievements, Challenges, and Opportunities. *arXiv preprint arXiv:2511.21382*, 2025. URL <https://arxiv.org/abs/2511.21382>.
- [37] Dekun Dai, Mingwei Liu, Anji Li, Jialun Cao, Yanlin Wang, Chong Wang, Xin Peng, and Zibin Zheng. FeedbackEval: A Benchmark for Evaluating Large Language Models in Feedback-Driven Code Repair Tasks. *arXiv preprint arXiv:2504.06939*, 2025. URL <https://arxiv.org/abs/2504.06939>.

- [38] Zhenlong Dai, Zhuoluo Zhao, Hengning Wang, Xiu Tang, Sai Wu, Chang Yao, Zhipeng Gao, and Jingyuan Chen. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. *arXiv preprint arXiv:2601.08545*, 2026. URL <https://arxiv.org/abs/2601.08545>.
- [39] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing. *arXiv preprint arXiv:2308.16557*, 2023. URL <https://arxiv.org/abs/2308.16557>.
- [40] David de Fitero-Dominguez, Eva Garcia-Lopez, Antonio Garcia-Cabot, and Jose-Javier Martinez-Herrais. Enhanced Automated Code Vulnerability Repair using Large Language Models. *arXiv preprint arXiv:2401.03741*, 2024. URL <https://arxiv.org/abs/2401.03741>.
- [41] David de Fitero-Dominguez, Antonio Garcia-Cabot, and Eva Garcia-Lopez. Self-Bootstrapping Automated Program Repair: Using LLMs to Generate and Evaluate Synthetic Training Data for Bug Repair. *arXiv preprint arXiv:2505.07372*, 2025. URL <https://arxiv.org/abs/2505.07372>.
- [42] Anderson de Lima Luiz. WARP – Web-Augmented Real-time Program Repairer: A Real-Time Compilation Error Resolution using LLMs and Web-Augmented Synthesis. *arXiv preprint arXiv:2509.25192*, 2025. URL <https://arxiv.org/abs/2509.25192>.
- [43] Meghdad Dehghan, Jie JW Wu, Fatemeh H. Fard, and Ali Ouni. MergeRepair: An Exploratory Study on Merging Task-Specific Adapters in Code LLMs for Automated Program Repair. *arXiv preprint arXiv:2408.09568*, 2024. URL <https://arxiv.org/abs/2408.09568>.
- [44] Amirhossein Deljouyi, Roham Koohestani, Maliheh Izadi, and Andy Zaidman. Leveraging Large Language Models for Enhancing the Understandability of Generated Unit Tests. *arXiv preprint arXiv:2408.11710*, 2024. URL <https://arxiv.org/abs/2408.11710>.
- [45] Yangruibo Ding, Jinjun Peng, Marcus J. Min, Gail Kaiser, Junfeng Yang, and Baishakhi Ray. Sem-Coder: Training Code Language Models with Comprehensive Semantics Reasoning. *arXiv preprint arXiv:2406.01006*, 2024. URL <https://arxiv.org/abs/2406.01006>.
- [46] Ion George Dinu, Marian Cristian Mihaescu, and Traian Rebedea. SmellBench: Evaluating LLM Agents on Architectural Code Smell Repair. *arXiv preprint arXiv:2605.07001*, 2026. URL <https://arxiv.org/abs/2605.07001>.
- [47] Xueying Du, Mingwei Liu, Hanlin Wang, Juntao Li, Xin Peng, and Yiling Lou. Exploring Large Language Models in Resolving Environment-Related Crash Bugs: Localizing and Repairing. *arXiv preprint arXiv:2312.10448*, 2023. URL <https://arxiv.org/abs/2312.10448>.
- [48] Vahid Etemadi and Gregorio Robles. SmartDoc: A Context-Aware Agentic Method Comment Generation Plugin. *arXiv preprint arXiv:2511.00450*, 2025. URL <https://arxiv.org/abs/2511.00450>.
- [49] Djire Alberick Euraste, Kabore Abdoul Kader, Jordan Samhi, Earl T. Barr, Jacques Klein, and Tegawende F. Bissyande. Learned or Memorized? Quantifying Memorization Advantage in Code LLMs. *arXiv preprint arXiv:2604.13997*, 2026. URL <https://arxiv.org/abs/2604.13997>.
- [50] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. Automated Repair of Programs from Large Language Models. *arXiv preprint arXiv:2205.10583*, 2022. URL <https://arxiv.org/abs/2205.10583>.
- [51] Zihan Fang, Yifan Zhang, Yueke Zhang, Kevin Leach, and Yu Huang. DPO-F+: Aligning Code Repair Feedback with Developers’ Preferences. *arXiv preprint arXiv:2511.01043*, 2025. URL <https://arxiv.org/abs/2511.01043>.
- [52] Mahdi Farzandway and Fatemeh Ghassemi. Automated Repair of C Programs Using Large Language Models. *arXiv preprint arXiv:2509.01947*, 2025. URL <https://arxiv.org/abs/2509.01947>.
- [53] Sakina Fatima, Hadi Hemmati, and Lionel Briand. FlakyFix: Using Large Language Models for Predicting Flaky Test Fix Categories and Test Code Repair. *arXiv preprint arXiv:2307.00012*, 2023. URL <https://arxiv.org/abs/2307.00012>.
- [54] Marcos Fuster-Pena, David de Fitero-Dominguez, Antonio Garcia-Cabot, and Eva Garcia-Lopez. RePaCA: Leveraging Reasoning Large Language Models for Static Automated Patch Correctness Assessment. *arXiv preprint arXiv:2507.22580*, 2025. URL <https://arxiv.org/abs/2507.22580>.
- [55] Varun Gadey, Zijie Liu, and Alexandra Dmitrienko. RAVEN: Agentic RAG for Automated Vulnerability Repair. *arXiv preprint arXiv:2606.22647*, 2026. URL <https://arxiv.org/abs/2606.22647>.
- [56] Shuzheng Gao, Eric John Li, Man Ho Lam, Jingyu Xiao, Yuxuan Wan, Chaozheng Wang, Ng Man Tik, and Michael R. Lyu. TREAT: A Code LLMs Trustworthiness / Reliability Evaluation and Testing Framework. *arXiv preprint arXiv:2510.17163*, 2025. URL <https://arxiv.org/abs/2510.17163>.
- [57] Xiang Gao, Yannic Noller, and Abhik Roychoudhury. Program Repair. *arXiv preprint*

- arXiv:2211.12787*, 2022. URL <https://arxiv.org/abs/2211.12787>.
- [58] Gwihwan Go, Quan Zhang, Chijin Zhou, Zhao Wei, and Yu Jiang. LSPRAG: LSP-Guided RAG for Language-Agnostic Real-Time Unit Test Generation. *arXiv preprint arXiv:2510.22210*, 2025. URL <https://arxiv.org/abs/2510.22210>.
- [59] Daniele Gorla, Shivam Kumar, Pietro Nicolaus Roselli Lorenzini, and Alireza Alipourfaz. CUBETESTERAI: Automated JUnit Test Generation using the LLaMA Model. *arXiv preprint arXiv:2504.15286*, 2025. URL <https://arxiv.org/abs/2504.15286>.
- [60] Sijia Gu, Noor Nashid, and Ali Mesbah. LLM Test Generation via Iterative Hybrid Program Analysis. *arXiv preprint arXiv:2503.13580*, 2025. URL <https://arxiv.org/abs/2503.13580>.
- [61] Go Frendi Gunawan and Mukhlis Amien. Comprehensive Evaluation of Large Language Models on Software Engineering Tasks: A Multi-Task Benchmark. *arXiv preprint arXiv:2602.07079*, 2026. URL <https://arxiv.org/abs/2602.07079>.
- [62] Hanyang Guo, Xiaoheng Xie, Hong-Ning Dai, Peng Di, Yu Zhang, Bishenghui Tao, and Zibin Zheng. Accelerating Automatic Program Repair with Dual Retrieval-Augmented Fine-Tuning and Patch Generation on Large Language Models. *arXiv preprint arXiv:2507.10103*, 2025. URL <https://arxiv.org/abs/2507.10103>.
- [63] Yaoqi Guo, Yang Liu, Jie M. Zhang, Yun Ma, Yiling Lou, and Zhenpeng Chen. SWE-Doctor: Guiding Software Engineering Agents with Runtime Diagnosis from Multi-Faceted Bug Reproduction Tests. *arXiv preprint arXiv:2607.00990*, 2026. URL <https://arxiv.org/abs/2607.00990>.
- [64] Yaoqi Guo, Ying Xiao, Jie M. Zhang, Mark Harman, Yiling Lou, Yang Liu, and Zhenpeng Chen. EET: Experience-Driven Early Termination for Cost-Efficient Software Engineering Agents. *arXiv preprint arXiv:2601.05777*, 2026. URL <https://arxiv.org/abs/2601.05777>.
- [65] Bethel Hall, Owen Ungaro, and William Eiers. CloudFix: Automated Policy Repair for Cloud Access Control Policies Using Large Language Models. *arXiv preprint arXiv:2512.09957*, 2025. URL <https://arxiv.org/abs/2512.09957>.
- [66] Mirazul Haque, Petr Babkin, Farima Farmahinifarahani, and Manuela Veloso. Towards Effectively Leveraging Execution Traces for Program Repair with Code LLMs. *arXiv preprint arXiv:2505.04441*, 2025. URL <https://arxiv.org/abs/2505.04441>.
- [67] Vahid Haratian, Atakan Akar, Berk Cakar, and Eray Tuzun. PR-Aware Automated Unit Test Generation: Challenges and Opportunities. *arXiv preprint arXiv:2605.25285*, 2026. URL <https://arxiv.org/abs/2605.25285>.
- [68] Kaivalya Hariharan, Uzay Girit, Atticus Wang, and Jacob Andreas. Breakpoint: Scalable evaluation of system-level reasoning in LLM code agents. *arXiv preprint arXiv:2506.00172*, 2025. URL <https://arxiv.org/abs/2506.00172>.
- [69] Sabaat Haroon, Mohammad Taha Khan, and Muhammad Ali Gulzar. Evaluating LLM-Based Test Generation Under Software Evolution. *arXiv preprint arXiv:2603.23443*, 2026. URL <https://arxiv.org/abs/2603.23443>.
- [70] Alif Al Hasan and Sumon Biswas. What Breaks When LLMs Code? Characterizing Operational Safety Failures of Agentic Code Assistants. *arXiv preprint arXiv:2605.30777*, 2026. URL <https://arxiv.org/abs/2605.30777>.
- [71] Xinyi He, Qian Liu, Mingzhe Du, Lin Yan, Zhijie Fan, Yiming Huang, Yin Zheng, Zejian Yuan, and Zejun Ma. SWE-Perf: Can Language Models Optimize Code Performance on Real-World Repositories? *arXiv preprint arXiv:2507.12415*, 2025. URL <https://arxiv.org/abs/2507.12415>.
- [72] Xu He, Shu Wang, and Kun Sun. PathFix: Automated Program Repair with Expected Path. *arXiv preprint arXiv:2510.14341*, 2025. URL <https://arxiv.org/abs/2510.14341>.
- [73] Yifeng He, Jiabo Huang, Yuyang Rong, Yiwen Guo, Ethan Wang, and Hao Chen. UniTSyn: A Large-Scale Dataset Capable of Enhancing the Prowess of Large Language Models for Program Testing. *arXiv preprint arXiv:2402.03396*, 2024. URL <https://arxiv.org/abs/2402.03396>.
- [74] Yifeng He, Jicheng Wang, Yuyang Rong, and Hao Chen. FuzzAug: Data Augmentation by Coverage-guided Fuzzing for Neural Test Generation. *arXiv preprint arXiv:2406.08665*, 2024. URL <https://arxiv.org/abs/2406.08665>.
- [75] David Hidvegi, Khashayar Etemadi, Sofia Bobadilla, and Martin Monperrus. CigaR: Cost-efficient Program Repair with LLMs. *arXiv preprint arXiv:2402.06598*, 2024. URL <https://arxiv.org/abs/2402.06598>.
- [76] Soneya Binta Hossain, Nan Jiang, Qiang Zhou, Xiaopeng Li, Wen-Hao Chiang, Yingjun Lyu, Hoan Nguyen, and Omer Tripp. A Deep Dive into Large Language Models for Automated Bug Localization and Repair. *arXiv preprint arXiv:2404.11595*, 2024. URL <https://arxiv.org/abs/2404.11595>.
- [77] Haichuan Hu, Ye Shang, Weifeng Sun, and Quan-jun Zhang. PSearch: Search-based Patch Generation in the Era of LLM-based Automated Program Repair. *arXiv preprint arXiv:2507.01827*, 2025. URL <https://arxiv.org/abs/2507.01827>.

- [78] Jia Hu, Youcheng Sun, and Pierre Olivier. Compartmentalization-Aware Automated Program Repair. *arXiv preprint arXiv:2603.09544*, 2026. URL <https://arxiv.org/abs/2603.09544>.
- [79] Junwei Hu, Weicheng Zheng, Yihan Liu, and Yan Liu. Optimizing Token Consumption in LLMs: A Nano Surge Approach for Code Reasoning Efficiency. *arXiv preprint arXiv:2504.15989*, 2025. URL <https://arxiv.org/abs/2504.15989>.
- [80] Dong Huang, Jie M. Zhang, Mark Harman, Qianru Zhang, Mingzhe Du, and See-Kiong Ng. Benchmarking LLMs for Unit Test Generation from Real-World Functions. *arXiv preprint arXiv:2508.00408*, 2025. URL <https://arxiv.org/abs/2508.00408>.
- [81] Jiangping Huang, Wenguang Ye, Weisong Sun, Jian Zhang, Mingyue Zhang, and Yang Liu. TraceCoder: A Trace-Driven Multi-Agent Framework for Automated Debugging of LLM-Generated Code. *arXiv preprint arXiv:2602.06875*, 2026. URL <https://arxiv.org/abs/2602.06875>.
- [82] Kai Huang, Zhengzi Xu, Su Yang, Hongyu Sun, Xuejun Li, Zheng Yan, and Yuqing Zhang. A Survey on Automated Program Repair Techniques. *arXiv preprint arXiv:2303.18184*, 2023. URL <https://arxiv.org/abs/2303.18184>.
- [83] Li Huang, Ilgiz Mustafin, Marco Piccioni, Alessandro Schena, Reto Weber, and Bertrand Meyer. Do AI models help produce verified bug fixes? *arXiv preprint arXiv:2507.15822*, 2025. URL <https://arxiv.org/abs/2507.15822>.
- [84] Zhili Huang, Ling Xu, Chao Liu, Weifeng Sun, Xu Zhang, Yan Lei, Meng Yan, and Hongyu Zhang. DynaFix: Iterative Automated Program Repair Driven by Execution-Level Dynamic Information. *arXiv preprint arXiv:2512.24635*, 2025. URL <https://arxiv.org/abs/2512.24635>.
- [85] Nafis Tanveer Islam, Joseph Khoury, Andrew Seong, Elias Bou-Harb, and Peyman Najafirad. Enhancing Source Code Security with LLMs: Demystifying The Challenges and Generating Reliable Repairs. *arXiv preprint arXiv:2409.00571*, 2024. URL <https://arxiv.org/abs/2409.00571>.
- [86] Raj Jaiswal, Anany Singh Divy, Savar Bhasin, Adi Bajpai, Tanuja Ganu, and Rajiv Ratn Shah. Obey, Diverge, Collapse: Blind Obedience to Incorrect Instructions Drives Code LLMs to Irrecoverable Code Semantic Collapse. *arXiv preprint arXiv:2607.04537*, 2026. URL <https://arxiv.org/abs/2607.04537>.
- [87] Suhwan Ji, Sanghwa Lee, Changsup Lee, Hyeon-seung Im, and Yo-Sub Han. Impact of Large Language Models of Code on Fault Localization. *arXiv preprint arXiv:2408.09657*, 2024. URL <https://arxiv.org/abs/2408.09657>.
- [88] Haoxiang Jia, Earl T. Barr, and Sergey Mechtaev. Compressing Code Context for LLM-based Issue Resolution. *arXiv preprint arXiv:2603.28119*, 2026. URL <https://arxiv.org/abs/2603.28119>.
- [89] Jiajun Jiang, Fengjie Li, Zijie Zhao, Zhirui Ye, Mengjiao Liu, Bo Wang, Hongyu Zhang, and Junjie Chen. Boosting Redundancy-based Automated Program Repair by Fine-grained Pattern Mining. *arXiv preprint arXiv:2312.15955*, 2023. URL <https://arxiv.org/abs/2312.15955>.
- [90] Nan Jiang, Qi Li, Lin Tan, and Tianyi Zhang. Collu-Bench: A Benchmark for Predicting Language Model Hallucinations in Code. *arXiv preprint arXiv:2410.09997*, 2024. URL <https://arxiv.org/abs/2410.09997>.
- [91] Yanjie Jiang, Yian Huang, Guancheng Wang, Junjie Chen, Hui Liu, and Lionel Briand. Characterizing the Failure Modes of LLMs in Resolving Real-World GitHub Issues. *arXiv preprint arXiv:2605.12270*, 2026. URL <https://arxiv.org/abs/2605.12270>.
- [92] Yilei Jiang, Jinyuan Hu, Qianyin Xiao, Yaozhi Zheng, Ruize Ma, Kaituo Feng, Jiaming Han, Tianshuo Peng, Kaixuan Fan, Manyuan Zhang, and Xiangyu Yue. OpenGame: Open Agentic Coding for Games. *arXiv preprint arXiv:2604.18394*, 2026. URL <https://arxiv.org/abs/2604.18394>.
- [93] Zhonghao Jiang, David Lo, and Zhongxin Liu. Agentic Software Issue Resolution with Large Language Models: A Survey. *arXiv preprint arXiv:2512.22256*, 2025. URL <https://arxiv.org/abs/2512.22256>.
- [94] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. InferFix: End-to-End Program Repair with LLMs. *arXiv preprint arXiv:2303.07263*, 2023. URL <https://arxiv.org/abs/2303.07263>.
- [95] Aliaksei Kaliutau. Autonomous Issue Resolver: Towards Zero-Touch Code Maintenance. *arXiv preprint arXiv:2512.08492*, 2025. URL <https://arxiv.org/abs/2512.08492>.
- [96] Sungmin Kang, Juyeon Yoon, and Shin Yoo. Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction. *arXiv preprint arXiv:2209.11515*, 2022. URL <https://arxiv.org/abs/2209.11515>.
- [97] Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. Explainable Automated Debugging via Large Language Model-driven Scientific Debugging. *arXiv preprint arXiv:2304.02195*, 2023. URL <https://arxiv.org/abs/2304.02195>.
- [98] Sungmin Kang, Haifeng Ruan, and Abhik Roychoudhury. AutoCodeSherpa: Symbolic Explanations in AI Coding Agents. *arXiv preprint*

- arXiv:2507.22414*, 2025. URL <https://arxiv.org/abs/2507.22414>.
- [99] Rabimba Karanjai, Lei Xu, and Weidong Shi. HPCAgentTester: A Multi-Agent LLM Approach for Enhanced HPC Unit Test Generation. *arXiv preprint arXiv:2511.10860*, 2025. URL <https://arxiv.org/abs/2511.10860>.
 - [100] Priyanka Kargupta, Ishika Agarwal, Dilek Hakkani-Tur, and Jiawei Han. Instruct, Not Assist: LLM-based Multi-Turn Planning and Hierarchical Questioning for Socratic Code Debugging. *arXiv preprint arXiv:2406.11709*, 2024. URL <https://arxiv.org/abs/2406.11709>.
 - [101] Changxin Ke, Rui Zhang, Jiaming Guo, Yuanbo Wen, Li Ding, Shuo Wang, Xuyuan Zhu, Xiong Peng, Di Huang, Zidong Du, Xing Hu, Qi Guo, and Yunji Chen. QiMeng-PRepair: Precise Code Repair via Edit-Aware Reward Optimization. *arXiv preprint arXiv:2604.05963*, 2026. URL <https://arxiv.org/abs/2604.05963>.
 - [102] Jiaolong Kong, Mingfei Cheng, Xiaofei Xie, Shangqing Liu, Xiaoning Du, and Qi Guo. ContrastRepair: Enhancing Conversation-Based Automated Program Repair via Contrastive Test Case Pairs. *arXiv preprint arXiv:2403.01971*, 2024. URL <https://arxiv.org/abs/2403.01971>.
 - [103] Jiaolong Kong, Xiaofei Xie, Yiheng Xiong, Yuekun Wang, and Jian Wang. Enhancing Automated Program Repair via Faulty Token Localization and Quality-Aware Patch Refinement. *arXiv preprint arXiv:2511.18001*, 2025. URL <https://arxiv.org/abs/2511.18001>.
 - [104] Milan De Koning, Ali Asgari, Pouria Derakhshanfar, and Annibale Panichella. A Metamorphic Testing Approach to Diagnosing Memorization in LLM-Based Program Repair. *arXiv preprint arXiv:2604.21579*, 2026. URL <https://arxiv.org/abs/2604.21579>.
 - [105] Michael Konstantinou, Renzo Degiovanni, and Mike Papadakis. How well LLM-based test generation techniques perform with newer LLM versions? *arXiv preprint arXiv:2601.09695*, 2026. URL <https://arxiv.org/abs/2601.09695>.
 - [106] Michael Konstantinou, Florian Tambon, and Mike Papadakis. On the risk of coding before testing: An empirical study on LLM-based test generation workflow. *arXiv preprint arXiv:2607.05139*, 2026. URL <https://arxiv.org/abs/2607.05139>.
 - [107] Charles Koutchme, Nicola Dainese, Sami Sarsa, Juho Leinonen, Arto Hellas, and Paul Denny. Benchmarking Educational Program Repair. *arXiv preprint arXiv:2405.05347*, 2024. URL <https://arxiv.org/abs/2405.05347>.
 - [108] Shiqi Kuang, Zhao Tian, Kaiwei Lin, Chaofan Tao, Shaowei Wang, Haoli Bai, Lifeng Shang, and Junjie Chen. REAgent: Requirement-Driven LLM Agents for Software Issue Resolution. *arXiv preprint arXiv:2604.06861*, 2026. URL <https://arxiv.org/abs/2604.06861>.
 - [109] Pragati Kumari and Novarun Deb. Sustainability Analysis of Prompt Strategies for SLM-based Automated Test Generation. *arXiv preprint arXiv:2604.02761*, 2026. URL <https://arxiv.org/abs/2604.02761>.
 - [110] Oka Kurniawan, Erick Chandra, Christopher M. Poskitt, Yannic Noller, Kenny Tsu Wei Choo, and Cyrille Jegourel. Designing for Novice Debuggers: A Pilot Study on an AI-Assisted Debugging Tool. *arXiv preprint arXiv:2509.21067*, 2025. URL <https://arxiv.org/abs/2509.21067>.
 - [111] Kazuki Kusama, Honglin Shu, Masanari Kondo, and Yasutaka Kamei. How Small is Enough? Empirical Evidence of Quantized Small Language Models for Automated Program Repair. *arXiv preprint arXiv:2508.16499*, 2025. URL <https://arxiv.org/abs/2508.16499>.
 - [112] Cuong Chi Le, Cuong Duc Van, Tung Duy Vu, Thai Minh Pham Vu, Hoang Nhat Phan, Huy Nhat Phan, and Tien N. Nguyen. TestWeaver: Execution-aware, Feedback-driven Regression Testing Generation with Large Language Models. *arXiv preprint arXiv:2508.01255*, 2025. URL <https://arxiv.org/abs/2508.01255>.
 - [113] Cuong Chi Le, Minh Le-Anh, Cuong Duc Van, and Tien N. Nguyen. Semantic Evolution over Populations for LLM-Guided Automated Program Repair. *arXiv preprint arXiv:2604.02134*, 2026. URL <https://arxiv.org/abs/2604.02134>.
 - [114] Tan Khang Le, Saba Alimadadi, and Steven Y. Ko. A Study of Vulnerability Repair in JavaScript Programs with Large Language Models. *arXiv preprint arXiv:2403.13193*, 2024. URL <https://arxiv.org/abs/2403.13193>.
 - [115] Tri Le, Thien Tran, Duy Cao, Vy Le, Tien Nguyen, and Vu Nguyen. KAT: Dependency-aware Automated API Testing with Large Language Models. *arXiv preprint arXiv:2407.10227*, 2024. URL <https://arxiv.org/abs/2407.10227>.
 - [116] Minh Le-Anh, Cuong Chi Le, and Tien N. Nguyen. Enhancing Program Repair with Specification Guidance and Intermediate Behavioral Signals. *arXiv preprint arXiv:2604.11770*, 2026. URL <https://arxiv.org/abs/2604.11770>.
 - [117] Minh Le-Anh, Cuong Chi Le, and Tien N. Nguyen. Teaching Code LLMs to Reason with Intermediate Formal Specifications. *arXiv preprint arXiv:2607.04232*, 2026. URL <https://arxiv.org/abs/2607.04232>.
 - [108] Shiqi Kuang, Zhao Tian, Kaiwei Lin, Chaofan

- [118] Thanh Le-Cong, Bach Le, and Toby Murray. Memory-Efficient Large Language Models for Program Repair with Semantic-Guided Patch Generation. *arXiv preprint arXiv:2410.16655*, 2024. URL <https://arxiv.org/abs/2410.16655>.
- [119] Cheryl Lee, Chunqiu Steven Xia, Longji Yang, Jentse Huang, Zhouruixin Zhu, Lingming Zhang, and Michael R. Lyu. UniDebugger: Hierarchical Multi-Agent Framework for Unified Software Debugging. *arXiv preprint arXiv:2404.17153*, 2024. URL <https://arxiv.org/abs/2404.17153>.
- [120] Kyla H. Levin, Nicolas van Kempen, Emery D. Berger, and Stephen N. Freund. ChatDBG: Augmenting Debugging with Large Language Models. *arXiv preprint arXiv:2403.16354*, 2024. URL <https://arxiv.org/abs/2403.16354>.
- [121] Fengjie Li, Jiajun Jiang, Jiajun Sun, and Hongyu Zhang. Hybrid Automated Program Repair by Combining Large Language Models and Program Analysis. *arXiv preprint arXiv:2406.00992*, 2024. URL <https://arxiv.org/abs/2406.00992>.
- [122] Guochang Li, Chen Zhi, Jialiang Chen, Junxiao Han, and Shuiguang Deng. Exploring Parameter-Efficient Fine-Tuning of Large Language Model on Automated Program Repair. *arXiv preprint arXiv:2406.05639*, 2024. URL <https://arxiv.org/abs/2406.05639>.
- [123] Han Li, Yuling Shi, Shaoxin Lin, Xiaodong Gu, Heng Lian, Xin Wang, Yantao Jia, Tao Huang, and Qianxiang Wang. SWE-Debate: Competitive Multi-Agent Debate for Software Issue Resolution. *arXiv preprint arXiv:2507.23348*, 2025. URL <https://arxiv.org/abs/2507.23348>.
- [124] KeFan Li, Mengfei Wang, Hengzhi Zhang, Zhichao Li, Yuan Yuan, Mu Li, Xiang Gao, Hailong Sun, Chunming Hu, and Weifeng Lv. InfCode: Adversarial Iterative Refinement of Tests and Patches for Reliable Software Issue Resolution. *arXiv preprint arXiv:2511.16004*, 2025. URL <https://arxiv.org/abs/2511.16004>.
- [125] Xiaoqi Li, Shipeng Ye, Wenkai Li, and Zongwei Li. SCPatcher: Automated Smart Contract Code Repair via Retrieval-Augmented Generation and Knowledge Graph. *arXiv preprint arXiv:2604.00687*, 2026. URL <https://arxiv.org/abs/2604.00687>.
- [126] Xingyu Li, Juefei Pu, Yifan Wu, Xiaochen Zou, Shitong Zhu, Xiaochen Zou, Shitong Zhu, Qiushi Wu, Zheng Zhang, Joshua Hsu, Yue Dong, Zhiyun Qian, Kangjie Lu, Trent Jaeger, Michael De Lucia, and Srikanth V. Krishnamurthy. What Do They Fix? LLM-Aided Categorization of Security Patches for Critical Memory Bugs. *arXiv preprint arXiv:2509.22796*, 2025. URL <https://arxiv.org/abs/2509.22796>.
- [127] Yixuan Liang. Monte Carlo Tree Search for Execution-Guided Program Repair with Large Language Models. *arXiv preprint arXiv:2602.00129*, 2026. URL <https://arxiv.org/abs/2602.00129>.
- [128] Dianshu Liao, Xin Yin, Shidong Pan, Chao Ni, Zhenchang Xing, and Xiaoyu Sun. Navigating the Labyrinth: Path-Sensitive Unit Test Generation with Large Language Models. *arXiv preprint arXiv:2509.23812*, 2025. URL <https://arxiv.org/abs/2509.23812>.
- [129] Fang Liu, Zhenwei Liu, Qianhui Zhao, Jing Jiang, Li Zhang, Ge Li, Zian Sun, Zhongqi Li, and Yuchi Ma. FastFixer: An Efficient and Effective Approach for Repairing Programming Assignments. *arXiv preprint arXiv:2410.21285*, 2024. URL <https://arxiv.org/abs/2410.21285>.
- [130] Jing Liu, Seongmin Lee, Eleonora Losiouk, and Marcel Bohme. Evaluating LLM-Based Regression Test Generation. *arXiv preprint arXiv:2501.11086*, 2025. URL <https://arxiv.org/abs/2501.11086>.
- [131] Quanming Liu, Xupeng Bu, Zhichao Yan, and Ru Li. \$T^3\$: Multi-level Tree-based Automatic Program Repair with Large Language Models. *arXiv preprint arXiv:2506.21211*, 2025. URL <https://arxiv.org/abs/2506.21211>.
- [132] Shukai Liu, Linzheng Chai, Jian Yang, Jiajun Shi, He Zhu, Liran Wang, Ke Jin, Wei Zhang, Hualei Zhu, Shuyue Guo, Tao Sun, Jiaheng Liu, Yunlong Duan, Yu Hao, Liqun Yang, Guanglin Niu, Ge Zhang, and Zhoujun Li. MdEval: Massively Multilingual Code Debugging. *arXiv preprint arXiv:2411.02310*, 2024. URL <https://arxiv.org/abs/2411.02310>.
- [133] Shunyu Liu, Guangdong Bai, Mark Utting, and Guowei Yang. RelRepair: Enhancing Automated Program Repair by Retrieving Relevant Code. *arXiv preprint arXiv:2509.16701*, 2025. URL <https://arxiv.org/abs/2509.16701>.
- [134] Xinyu Liu, Jiayu Ren, Yusen Wang, Qi Xin, Xiaoyuan Xie, and Jifeng Xuan. SiblingRepair: Sibling-Based Multi-Hunk Repair with Large Language Models. *arXiv preprint arXiv:2605.06209*, 2026. URL <https://arxiv.org/abs/2605.06209>.
- [135] Yizhou Liu, Pengfei Gao, Xinchun Wang, Jie Liu, Yexuan Shi, Zhao Zhang, and Chao Peng. MarsCode Agent: AI-native Automated Bug Fixing. *arXiv preprint arXiv:2409.00899*, 2024. URL <https://arxiv.org/abs/2409.00899>.
- [136] Zhengyao Liu, Yunlong Ma, Jingxuan Xu, Junchen Ai, Xiang Gao, Hailong Sun, and Abhik Roychoudhury. Agent That Debugs: Dynamic State-Guided Vulnerability Repair. *arXiv preprint arXiv:2504.07634*, 2025. URL <https://arxiv.org/abs/2504.07634>.

- [137] Zhuoyao Liu, Zhengran Zeng, Shu-Dong Huang, Yang Liu, Shikun Zhang, and Wei Ye. GALA: Multimodal Graph Alignment for Bug Localization in Automated Program Repair. *arXiv preprint arXiv:2604.08089*, 2026. URL <https://arxiv.org/abs/2604.08089>.
- [138] Andrea Lops, Fedelucio Narducci, Azzurra Ragone, Michelantonio Trizio, and Claudio Bartolini. A System for Automated Unit Test Generation Using Large Language Models and Assessment of Generated Test Suites. *arXiv preprint arXiv:2408.07846*, 2024. URL <https://arxiv.org/abs/2408.07846>.
- [139] Andrea Lops, Fedelucio Narducci, Azzurra Ragone, Michelantonio Trizio, and Claudio Bartolini. LLMs for Automated Unit Test Generation and Assessment in Java: The AgoneTest Framework. *arXiv preprint arXiv:2511.20403*, 2025. URL <https://arxiv.org/abs/2511.20403>.
- [140] Chiwang Luk, Matin Mohammad Najafi, Zhifeng Jia, Wei Yang, Xiuchang Li, Jinwei Zhu, Yang Ren, Lei Chen, and Gao Cong. ContextSniper: AntTrail’s Token-Efficient Code Memory for Repository-Level Program Repair. *arXiv preprint arXiv:2607.01916*, 2026. URL <https://arxiv.org/abs/2607.01916>.
- [141] Wenqiang Luo, Jacky Wai Keung, Boyang Yang, He Ye, Claire Le Goues, Tegawende F. Bissyande, Haoye Tian, and Bach Le. When Fine-Tuning LLMs Meets Data Privacy: An Empirical Study of Federated Learning in LLM-Based Program Repair. *arXiv preprint arXiv:2412.01072*, 2024. URL <https://arxiv.org/abs/2412.01072>.
- [142] Zizhang Luo, Fan Cui, Kexing Zhou, Runlin Guo, Mile Xia, Hongyuan Hou, and Yun Liang. R3A: Reliable RTL Repair Framework with Multi-Agent Fault Localization and Stochastic Tree-of-Thoughts Patch Generation. *arXiv preprint arXiv:2511.20090*, 2025. URL <https://arxiv.org/abs/2511.20090>.
- [143] Michael R. Lyu, Baishakhi Ray, Abhik Roychoudhury, Shin Hwei Tan, and Patanamon Thongtanunam. Automatic Programming: Large Language Models and Beyond. *arXiv preprint arXiv:2405.02213*, 2024. URL <https://arxiv.org/abs/2405.02213>.
- [144] Yinghang Ma, Jiho Shin, Leuson Da Silva, Zhen Ming, Jiang, Song Wang, Foutse Khomh, and Shin Hwei Tan. BloomAPR: A Bloom’s Taxonomy-based Framework for Assessing the Capabilities of LLM-Powered APR Solutions. *arXiv preprint arXiv:2509.25465*, 2025. URL <https://arxiv.org/abs/2509.25465>.
- [145] Roman Machacek, Anastasiia Grishina, Max Hort, and Leon Moonen. The Impact of Fine-tuning Large Language Models on Automated Program Repair. *arXiv preprint arXiv:2507.19909*, 2025. URL <https://arxiv.org/abs/2507.19909>.
- [146] Md Afif Al Mamun and Gias Uddin. BLA-agent: Agentic RAG for File-Level Bug Localization. *arXiv preprint arXiv:2605.17965*, 2026. URL <https://arxiv.org/abs/2605.17965>.
- [147] Dung Nguyen Manh, Thang Phan Chau, Nam Le Hai, Thong T. Doan, Nam V. Nguyen, Quang Pham, and Nghi D. Q. Bui. CodeMMLU: A Multi-Task Benchmark for Assessing Code Understanding & Reasoning Capabilities of CodeLLMs. *arXiv preprint arXiv:2410.01999*, 2024. URL <https://arxiv.org/abs/2410.01999>.
- [148] Chenhui Mao, Yuanting Lei, Zhixiang Wei, Ming Liang, Zhixiang Wang, Jingxuan Xu, Dajun Chen, Wei Jiang, and Yong Li. EGSS: Entropy-guided Stepwise Scaling for Reliable Software Engineering. *arXiv preprint arXiv:2602.05242*, 2026. URL <https://arxiv.org/abs/2602.05242>.
- [149] Matias Martinez and Xavier Franch. Dissecting the SWE-Bench Leaderboards: Profiling Submitters and Architectures of LLM- and Agent-Based Repair Systems. *arXiv preprint arXiv:2506.17208*, 2025. URL <https://arxiv.org/abs/2506.17208>.
- [150] Matias Martinez and Xavier Franch. What’s in a Benchmark? The Case of SWE-Bench in Automated Program Repair. *arXiv preprint arXiv:2602.04449*, 2026. URL <https://arxiv.org/abs/2602.04449>.
- [151] Matias Martinez, Silverio Martinez-Fernandez, and Xavier Franch. Energy Consumption of Automated Program Repair. *arXiv preprint arXiv:2211.12104*, 2022. URL <https://arxiv.org/abs/2211.12104>.
- [152] Spencer Mateega, Jeff Yang, Tiana Costello, Shaurya Jadhav, Nicole Tian, and Agustin Garcinuno. IDE-Bench: Evaluating Large Language Models as IDE Agents on Real-World Software Engineering Tasks. *arXiv preprint arXiv:2601.20886*, 2026. URL <https://arxiv.org/abs/2601.20886>.
- [153] Xiangxin Meng, Zexiong Ma, Pengfei Gao, and Chao Peng. An Empirical Study on LLM-based Agents for Automated Bug Fixing. *arXiv preprint arXiv:2411.10213*, 2024. URL <https://arxiv.org/abs/2411.10213>.
- [154] Edward Misback, Erik Vank, Zachary Tatlock, and Steven Tanimoto. Codetations: Intelligent, Persistent Notes and UIs for Programs and Other Documents. *arXiv preprint arXiv:2504.18702*, 2025. URL <https://arxiv.org/abs/2504.18702>.
- [155] Mohammad Mahdi Mohajer, Reem Aleithan, Nima Shiri Harzevili, Moshi Wei, Alvine Boaye Belle, Hung Viet Pham, and Song Wang. Skip-Analyzer: A Tool for Static Code Analysis

- with Large Language Models. *arXiv preprint arXiv:2310.18532*, 2023. URL <https://arxiv.org/abs/2310.18532>.
- [156] Takuro Morimoto and Harumi Haraguchi. A Study on the Improvement of Code Generation Quality Using Large Language Models Leveraging Product Documentation. *arXiv preprint arXiv:2503.17837*, 2025. URL <https://arxiv.org/abs/2503.17837>.
- [157] Sahand Moslemi, Mayasah Lami, and Anil Koyuncu. Historian: Reducing Manual Validation in APR Benchmarking via Evidence-Based Assessment. *arXiv preprint arXiv:2603.00649*, 2026. URL <https://arxiv.org/abs/2603.00649>.
- [158] Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. EXPEREPAIR: Dual-Memory Enhanced LLM-based Repository-Level Program Repair. *arXiv preprint arXiv:2506.10484*, 2025. URL <https://arxiv.org/abs/2506.10484>.
- [159] Niels Mundler, Mark Niklas Muller, Jingxuan He, and Martin Vechev. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. *arXiv preprint arXiv:2406.12952*, 2024. URL <https://arxiv.org/abs/2406.12952>.
- [160] Ansong Ni, Miltiadis Allamanis, Arman Cohan, Yinlin Deng, Kensen Shi, Charles Sutton, and Pengcheng Yin. NExT: Teaching Large Language Models to Reason about Code Execution. *arXiv preprint arXiv:2404.14662*, 2024. URL <https://arxiv.org/abs/2404.14662>.
- [161] Feifei Niu, Chuanyi Li, Kui Liu, Xin Xia, and David Lo. When Deep Learning Meets Information Retrieval-based Bug Localization: A Survey. *arXiv preprint arXiv:2505.00144*, 2025. URL <https://arxiv.org/abs/2505.00144>.
- [162] Yu Nong, Haoran Yang, Long Cheng, Hongxin Hu, and Haipeng Cai. APPATCH: Automated Adaptive Prompting Large Language Models for Real-World Software Vulnerability Patching. *arXiv preprint arXiv:2408.13597*, 2024. URL <https://arxiv.org/abs/2408.13597>.
- [163] Ali Nouri, Johan Andersson, Kailash De Jesus Hornig, Zhennan Fei, Emil Knabe, Hakan Sivencrona, Beatriz Cabrero-Daniel, and Christian Berger. On Simulation-Guided LLM-based Code Generation for Safe Autonomous Driving Software. *arXiv preprint arXiv:2504.02141*, 2025. URL <https://arxiv.org/abs/2504.02141>.
- [164] Daniel Ogenrwot and John Businge. PatchTrack: A Comprehensive Analysis of ChatGPT’s Influence on Pull Request Outcomes. *arXiv preprint arXiv:2505.07700*, 2025. URL <https://arxiv.org/abs/2505.07700>.
- [165] Pedro Orvalho, Mikolas Janota, and Vasco Manquinho. Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization. *arXiv preprint arXiv:2502.07786*, 2024. URL <https://arxiv.org/abs/2502.07786>.
- [166] Md Sultanul Islam Ovi, Nafisa Anjum, Tasmina Haque Bithe, Md. Mahabubur Rahman, and Mst. Shahnaj Akter Smrity. Benchmarking ChatGPT, Codeium, and GitHub Copilot: A Comparative Study of AI-Driven Programming and Debugging Assistants. *arXiv preprint arXiv:2409.19922*, 2024. URL <https://arxiv.org/abs/2409.19922>.
- [167] Anvith Pabba, Simin Chen, Alex Mathai, Anindya Chakraborty, and Baishakhi Ray. RE-FINE: Enhancing Program Repair Agents through Context-Aware Patch Refinement. *arXiv preprint arXiv:2510.03588*, 2025. URL <https://arxiv.org/abs/2510.03588>.
- [168] Anvith Pabba, Alex Mathai, Anindya Chakraborty, and Baishakhi Ray. SemAgent: A Semantics Aware Program Repair Agent. *arXiv preprint arXiv:2506.16650*, 2025. URL <https://arxiv.org/abs/2506.16650>.
- [169] Rangeet Pan, Myeongsoo Kim, Rahul Krishna, Raju Pavuluri, and Saurabh Sinha. ASTER: Natural and Multi-language Unit Test Generation with LLMs. *arXiv preprint arXiv:2409.03093*, 2024. URL <https://arxiv.org/abs/2409.03093>.
- [170] Rangeet Pan, Raju Pavuluri, Ruikai Huang, Rahul Krishna, Tyler Stennett, Alessandro Orso, and Saurabh Sinha. SAINT: Service-level Integration Test Generation with Program Analysis and LLM-based Agents. *arXiv preprint arXiv:2511.13305*, 2025. URL <https://arxiv.org/abs/2511.13305>.
- [171] Yue Pan, Zimin Chen, Siyu Lu, Zhaoyang Chu, Xiang Li, Han Li, Yang Feng, Claire Le Goues, Federica Sarro, Martin Monperrus, and He Ye. Prometheus: Towards Long-Horizon Codebase Navigation for Repository-Level Problem Solving. *arXiv preprint arXiv:2507.19942*, 2025. URL <https://arxiv.org/abs/2507.19942>.
- [172] Nikhil Parasaram, Huijie Yan, Boyu Yang, Zineb Flahy, Abriele Qudsi, Damian Ziaber, Earl Barr, and Sergey Mechtaev. The Fact Selection Problem in LLM-Based Program Repair. *arXiv preprint arXiv:2404.05520*, 2024. URL <https://arxiv.org/abs/2404.05520>.
- [173] Rishov Paul, Md. Mohib Hossain, Mohammed Latif Siddiq, Masum Hasan, Anindya Iqbal, and Joanna C. S. Santos. Enhancing Automated Program Repair through Fine-tuning and Prompt Engineering. *arXiv preprint arXiv:2304.07840*, 2023. URL <https://arxiv.org/abs/2304.07840>.
- [174] Wenjun Peng, Xinyu Wang, and Qi Wu. ProxyWar: Dynamic Assessment of LLM Code Generation in

- Game Arenas. *arXiv preprint arXiv:2602.04296*, 2026. URL <https://arxiv.org/abs/2602.04296>.
- [175] An B. B. Pham, Hoa T. Nguyen, and Muhammad Usman. QBugLM: An Agentic Benchmarking Framework for LLM-based Quantum Software Debugging. *arXiv preprint arXiv:2606.07314*, 2026. URL <https://arxiv.org/abs/2606.07314>.
- [176] Minh V. T. Pham, Huy N. Phan, Hoang N. Phan, Cuong Le Chi, Tien N. Nguyen, and Nghi D. Q. Bui. SWE-Synth: Synthesizing Verifiable Bug-Fix Data to Enable Large Language Models in Resolving Real-World Bugs. *arXiv preprint arXiv:2504.14757*, 2025. URL <https://arxiv.org/abs/2504.14757>.
- [177] Griffin Pitts, Aum Pandya, Darsh Rank, Tirth Bhatt, Muntasir Hoq, and Bitu Akram. Automated Program Repair of Uncompilable Student Code. *arXiv preprint arXiv:2510.06187*, 2025. URL <https://arxiv.org/abs/2510.06187>.
- [178] Laura Plein, Wendkuuni C. Ouedraogo, Jacques Klein, and Tegawende F. Bissyande. Automatic Generation of Test Cases based on Bug Reports: a Feasibility Study with Large Language Models. *arXiv preprint arXiv:2310.06320*, 2023. URL <https://arxiv.org/abs/2310.06320>.
- [179] Julius Porbeck, Christian Medeiros Adriano, and Holger Giese. From Program Slices to Causal Clarity: Evaluating Faithful, Actionable LLM-Generated Failure Explanations via Context Partitioning and LLM-as-a-Judge. *arXiv preprint arXiv:2604.18309*, 2026. URL <https://arxiv.org/abs/2604.18309>.
- [180] Piotr Przymus, Andreas Happe, and Jurgen Cito. Adversarial Bug Reports as a Security Risk in Language Model-Based Automated Program Repair. *arXiv preprint arXiv:2509.05372*, 2025. URL <https://arxiv.org/abs/2509.05372>.
- [181] Haomin Qi, Yuyang Du, Lihao Zhang, Soung Chang Liew, Kexin Chen, and Yining Du. VeriRAG: A Retrieval-Augmented Framework for Automated RTL Testability Repair. *arXiv preprint arXiv:2507.15664*, 2025. URL <https://arxiv.org/abs/2507.15664>.
- [182] Yixian Qi, Jiajun Jiang, Fengjie Li, Bowen Chen, Hongyu Zhang, and Junjie Chen. Improving Compiler Bug Isolation by Leveraging Large Language Models. *arXiv preprint arXiv:2506.17647*, 2025. URL <https://arxiv.org/abs/2506.17647>.
- [183] Irtaza Sajid Qureshi, Zhen Ming, and Jiang. Test Case Generation from Bug Reports via Large Language Models: A Cognitive Layered Evaluation Framework. *arXiv preprint arXiv:2510.05365*, 2025. URL <https://arxiv.org/abs/2510.05365>.
- [184] Fazle Rabbi and Jinqui Yang. HEJ-Robust: A Robustness Benchmark for LLM-Based Automated Program Repair. *arXiv preprint arXiv:2605.02215*, 2026. URL <https://arxiv.org/abs/2605.02215>.
- [185] Tasnia Rahman, Sathish A. P. Kumar, Sumit Jha, and Arvind Ramanathan. ACE-RLHF: Automated Code Evaluation and Socratic Feedback Generation Tool using Large Language Models and Reinforcement Learning with Human Feedback. *arXiv preprint arXiv:2504.04657*, 2025. URL <https://arxiv.org/abs/2504.04657>.
- [186] Ravi Raju, Mengmeng Ji, Shubhangi Upasani, Bo Li, and Urmish Thakker. The Limits of Long-Context Reasoning in Automated Bug Fixing. *arXiv preprint arXiv:2602.16069*, 2026. URL <https://arxiv.org/abs/2602.16069>.
- [187] Syed Md Mukit Rashid, Abdullah Al Ishtiaq, Kai Tu, Yilu Dong, Tianwei Wu, Ali Ranjbar, Tianchang Yang, Najrin Sultana, Shagufta Mehnaz, and Syed Rafiul Hussain. LogicEval: A Systematic Framework for Evaluating Automated Repair Techniques for Logical Vulnerabilities in Real-World Software. *arXiv preprint arXiv:2604.12994*, 2026. URL <https://arxiv.org/abs/2604.12994>.
- [188] Joseph Renzullo, Pemma Reiter, Westley Weimer, and Stephanie Forrest. Automated Program Repair: Emerging trends pose and expose problems for benchmarks. *arXiv preprint arXiv:2405.05455*, 2024. URL <https://arxiv.org/abs/2405.05455>.
- [189] Sujoy Roychowdhury, Giriprasad Sridhara, A K Raghavan, Joy Bose, Sourav Mazumdar, Hamender Singh, Srinivasan Bajji Sugumaran, and Ricardo Britto. Static Program Analysis Guided LLM Based Unit Test Generation. *arXiv preprint arXiv:2503.05394*, 2025. URL <https://arxiv.org/abs/2503.05394>.
- [190] Haifeng Ruan, Yuntong Zhang, and Abhik Roychoudhury. SpecRover: Code Intent Extraction via LLMs. *arXiv preprint arXiv:2408.02232*, 2024. URL <https://arxiv.org/abs/2408.02232>.
- [191] Fernando Vallecillos Ruiz, Anastasiia Grishina, Max Hort, and Leon Moonen. Assessing the Latent Automated Program Repair Capabilities of Large Language Models using Round-Trip Translation. *arXiv preprint arXiv:2401.07994*, 2024. URL <https://arxiv.org/abs/2401.07994>.
- [192] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. Code-Aware Prompting: A study of Coverage Guided Test Generation in Regression Setting using LLM. *arXiv preprint arXiv:2402.00097*, 2024. URL <https://arxiv.org/abs/2402.00097>.
- [193] Amirali Sajadi, Kostadin Damevski, and Preetha Chatterjee. How Safe Are AI-Generated Patches?

- A Large-scale Study on Security Risks in LLM and Agentic Automated Program Repair on SWE-bench. *arXiv preprint arXiv:2507.02976*, 2025. URL <https://arxiv.org/abs/2507.02976>.
- [194] Asif Mohammed Samir and Mohammad Masudur Rahman. Improving IR-based Bug Localization with Semantics-Driven Query Reduction. *arXiv preprint arXiv:2510.04468*, 2025. URL <https://arxiv.org/abs/2510.04468>.
- [195] Asif Mohammed Samir and Mohammad Masudur Rahman. Improved Bug Localization with AI Agents Leveraging Hypothesis and Dynamic Cognition. *arXiv preprint arXiv:2601.12522*, 2026. URL <https://arxiv.org/abs/2601.12522>.
- [196] E. G. Santana, Gabriel Benjamin, Melissa Araujo, Harrison Santos, David Freitas, Eduardo Almeida, Paulo Anselmo da M. S. Neto, Jiawei Li, Jina Chun, and Iftekhhar Ahmed. Which Prompting Technique Should I Use? An Empirical Investigation of Prompting Techniques for Software Engineering Tasks. *arXiv preprint arXiv:2506.05614*, 2025. URL <https://arxiv.org/abs/2506.05614>.
- [197] Max Schafer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *arXiv preprint arXiv:2302.06527*, 2023. URL <https://arxiv.org/abs/2302.06527>.
- [198] Melika Sepidband, Hung Viet Pham, and Hadi Hemmati. On the Role of Fault Localization Context for LLM-Based Program Repair. *arXiv preprint arXiv:2604.05481*, 2026. URL <https://arxiv.org/abs/2604.05481>.
- [199] Melika Sepidband, Hamed Taherkhani, Hung Viet Pham, and Hadi Hemmati. RGFL: Reasoning Guided Fault Localization for Automated Program Repair Using Large Language Models. *arXiv preprint arXiv:2601.18044*, 2026. URL <https://arxiv.org/abs/2601.18044>.
- [200] Kareem Shehada, Yifan Wu, Wyatt D. Feng, Adithya Iyer, Gryphon Kumpf, Yangruibo Ding, and Zhiyun Qian. Rethinking Kernel Program Repair: Benchmarking and Enhancing LLMs with RGym. *arXiv preprint arXiv:2511.15757*, 2025. URL <https://arxiv.org/abs/2511.15757>.
- [201] Yu Shi, Abdul Ali Bangash, Emad Fallahzadeh, Bram Adams, and Ahmed E. Hassan. HAFix: History-Augmented Large Language Models for Bug Fixing. *arXiv preprint arXiv:2501.09135*, 2025. URL <https://arxiv.org/abs/2501.09135>.
- [202] Yu Shi, Hao Li, Bram Adams, and Ahmed E. Hassan. HAFixAgent: History-Aware Program Repair Agent. *arXiv preprint arXiv:2511.01047*, 2025. URL <https://arxiv.org/abs/2511.01047>.
- [203] Andre Silva, Sen Fang, and Martin Monperrus. RepairLLaMA: Efficient Representations and Fine-Tuned Adapters for Program Repair. *arXiv preprint arXiv:2312.15698*, 2023. URL <https://arxiv.org/abs/2312.15698>.
- [204] Ananya Singha, Harshita Sahijwani, Walt Williams, Emmanuel Aboah Boateng, Nick Hausman, Miguel Di Luca, Keegan Choudhury, Chaya Binet, Vu Le, Tianwei Chen, Oryan Rokeah Chen, Sulaiman Vesal, and Sadid Hasan. Benchmark Dataset Generation and Evaluation for Excel Formula Repair with LLMs. *arXiv preprint arXiv:2508.11715*, 2025. URL <https://arxiv.org/abs/2508.11715>.
- [205] Ha Min Son, Huan Ren, Xin Liu, and Zhe Zhao. Automating Android Build Repair: Bridging the Reasoning-Execution Gap in LLM Agents with Domain-Specific Tools. *arXiv preprint arXiv:2510.08640*, 2025. URL <https://arxiv.org/abs/2510.08640>.
- [206] Benjamin Steenhoek, Michele Tufano, Neel Sundaresan, and Alexey Svyatkovskiy. Reinforcement Learning from Automatic Feedback for High-Quality Unit Test Generation. *arXiv preprint arXiv:2412.14308*, 2024. URL <https://arxiv.org/abs/2412.14308>.
- [207] Philipp Straubinger, Marvin Kreis, Stephan Lukasczyk, and Gordon Fraser. Mutation Testing via Iterative Large Language Model-Driven Scientific Debugging. *arXiv preprint arXiv:2503.08182*, 2025. URL <https://arxiv.org/abs/2503.08182>.
- [208] Jiajun Sun, Fengjie Li, Xinzhu Qi, Hongyu Zhang, and Jiajun Jiang. Empirical Evaluation of Large Language Models in Automated Program Repair. *arXiv preprint arXiv:2506.13186*, 2025. URL <https://arxiv.org/abs/2506.13186>.
- [209] Maolin Sun, Yibiao Yang, Xuanlin Liu, Yuming Zhou, and Baowen Xu. DebugHarness: Emulating Human Dynamic Debugging for Autonomous Program Repair. *arXiv preprint arXiv:2604.03610*, 2026. URL <https://arxiv.org/abs/2604.03610>.
- [210] Hamed Taherkhani, Alireza DaghighFarsoodeh, Mohammad Chowdhury, Hung Viet Pham, and Hadi Hemmati. Consistency Meets Verification: Enhancing Test Generation Quality in Large Language Models Without Ground-Truth Solutions. *arXiv preprint arXiv:2602.10522*, 2026. URL <https://arxiv.org/abs/2602.10522>.
- [211] Xiaoxuan Tang, Jincheng Wang, Liwei Luo, Jingxuan Xu, Sheng Zhou, Dajun Chen, Wei Jiang, and Yong Li. SVRepair: Structured Visual Reasoning for Automated Program Repair. *arXiv preprint arXiv:2602.06090*, 2026. URL <https://arxiv.org/abs/2602.06090>.
- [212] Yuheng Tang, Hongwei Li, Kaijie Zhu, Michael Yang, Yangruibo Ding, and Wenbo Guo. Co-

- PatcheR: Collaborative Software Patching with Component(s)-specific Small Reasoning Models. *arXiv preprint arXiv:2505.18955*, 2025. URL <https://arxiv.org/abs/2505.18955>.
- [213] Vali Tawosi, Salwa Alamir, Xiaomo Liu, and Manuela Veloso. Meta-RAG on Large Codebases Using Code Summarization. *arXiv preprint arXiv:2508.02611*, 2025. URL <https://arxiv.org/abs/2508.02611>.
- [214] Natchaya Temyingyong, Daman Jain, Neeraj Kumarsahu, Prabhat Kumar, Rachata Phondi, Wachiravit Modecrua, Krittanon Kaewtaee, Kritin Pachtrachai, and Touchapon Kraisingkorn. ROAD: Reflective Optimization via Automated Debugging for Zero-Shot Agent Alignment. *arXiv preprint arXiv:2512.24040*, 2025. URL <https://arxiv.org/abs/2512.24040>.
- [215] Matthew Thompson. The Dual-State Architecture for Reliable LLM Agents. *arXiv preprint arXiv:2512.20660*, 2025. URL <https://arxiv.org/abs/2512.20660>.
- [216] Muxin Tian, Zhe Wang, Blair Yang, Zhenwei Tang, Kunlun Zhu, Honghua Dong, Hanchen Li, Xinni Xie, Guangjing Wang, and Jiaxuan You. SWE-Bench Mobile: Can Large Language Model Agents Develop Industry-Level Mobile Applications? *arXiv preprint arXiv:2602.09540*, 2026. URL <https://arxiv.org/abs/2602.09540>.
- [217] Lukas Twist. Summary-Mediated Repair: Can LLMs use code summarisation as a tool for program repair? *arXiv preprint arXiv:2511.18782*, 2025. URL <https://arxiv.org/abs/2511.18782>.
- [218] Fernando Vallecillos-Ruiz, Giordano d’Aloisio, Max Hort, Luca Traini, Antiniscia Di Marco, and Leon Moonen. Smaller Models, Unexpected Costs: Trade-offs in LLM Quantization for Automated Program Repair. *arXiv preprint arXiv:2606.27205*, 2026. URL <https://arxiv.org/abs/2606.27205>.
- [219] Antonio Vitale, Emanuela Guglielmi, Simone Scalabrino, and Rocco Oliveto. On the Impact of Code Comments for Automated Bug-Fixing: An Empirical Study. *arXiv preprint arXiv:2601.23059*, 2026. URL <https://arxiv.org/abs/2601.23059>.
- [220] Maja Vukovic, Rangeet Pan, Tin Kam Ho, Rahul Krishna, Raju Pavuluri, and Michele Merler. Usage, Effects and Requirements for AI Coding Assistants in the Enterprise: An Empirical Study. *arXiv preprint arXiv:2601.20112*, 2026. URL <https://arxiv.org/abs/2601.20112>.
- [221] Jakub Walczak, Piotr Tomalak, and Artur Laskowski. Impact of Code Context and Prompting Strategies on Automated Unit Test Generation with Modern General-Purpose Large Language Models. *arXiv preprint arXiv:2507.14256*, 2025. URL <https://arxiv.org/abs/2507.14256>.
- [222] Che Wang, Jiashuo Zhang, Jianbo Gao, Libin Xia, Zhi Guan, and Zhong Chen. Contract-Tinker: LLM-Empowered Vulnerability Repair for Real-World Smart Contracts. *arXiv preprint arXiv:2409.09661*, 2024. URL <https://arxiv.org/abs/2409.09661>.
- [223] Guancheng Wang, Qinghua Xu, Lionel Briand, and Kui Liu. Mutation-Guided Unit Test Generation with a Large Language Model. *arXiv preprint arXiv:2506.02954*, 2025. URL <https://arxiv.org/abs/2506.02954>.
- [224] Guancheng Wang, Qinghua Xu, Lionel C. Briand, Zhaoqiang Guo, and Kui Liu. Call-Chain-Aware LLM-Based Test Generation for Java Projects. *arXiv preprint arXiv:2604.22046*, 2026. URL <https://arxiv.org/abs/2604.22046>.
- [225] Jialin Wang and Zhihua Duan. Empirical Research on Utilizing LLM-based Agents for Automated Bug Fixing via LangGraph. *arXiv preprint arXiv:2502.18465*, 2025. URL <https://arxiv.org/abs/2502.18465>.
- [226] Jian Wang, Xiaofei Xie, Qiang Hu, Shangqing Liu, Jiongchi Yu, Jiaolong Kong, and Yi Li. Defects4C: Benchmarking Large Language Model Repair Capability with C/C++ Bugs. *arXiv preprint arXiv:2510.11059*, 2025. URL <https://arxiv.org/abs/2510.11059>.
- [227] Jiexin Wang, Xitong Luo, Liuwen Cao, Hongkui He, Hailin Huang, Jiayuan Xie, Adam Jatowt, and Yi Cai. Is Your AI-Generated Code Really Safe? Evaluating Large Language Models on Secure Code Generation with CodeSecEval. *arXiv preprint arXiv:2407.02395*, 2024. URL <https://arxiv.org/abs/2407.02395>.
- [228] Lilin Wang, Lucas Ramalho, Alan Celestino, Phuc Anthony Pham, Yu Liu, Umang Kumar Sinha, Andres Portillo, Onassis Osunwa, and Gabriel Maduekwe. SWE-Bench++: A Framework for the Scalable Generation of Software Engineering Benchmarks from Open-Source Repositories. *arXiv preprint arXiv:2512.17419*, 2025. URL <https://arxiv.org/abs/2512.17419>.
- [229] Sinan Wang, Zhiyuan Zhong, Shaojin Wen, and Yepang Liu. Multi-agent Assisted Automatic Test Generation for Java JSON Libraries. *arXiv preprint arXiv:2410.09414*, 2024. URL <https://arxiv.org/abs/2410.09414>.
- [230] Yibo Wang, Congying Xia, Wenting Zhao, Jiangshu Du, Chunyu Miao, Zhongfen Deng, Philip S. Yu, and Chen Xing. MultiFileTest: A Multi-File-Level LLM Unit Test Generation Benchmark and Impact of Error Fixing Mechanisms. *arXiv preprint arXiv:2502.06556*, 2025. URL <https://arxiv.org/abs/2502.06556>.

- [231] Yifei Wang, Tianlin Li, Xiaohan Zhang, Xiaoyu Zhang, Wei Ma, Mingfei Cheng, and Li Pan. Hidden Reliability Risks in Large Language Models: Systematic Identification of Precision-Induced Output Disagreements. *arXiv preprint arXiv:2604.19790*, 2026. URL <https://arxiv.org/abs/2604.19790>.
- [232] Yihan Wang, Cheng Liu, Jiazheng Zhang, Lei Zhang, Long Cheng, Xiaowei Li, and Huawei Li. VeriPilot: An LLM-Powered Verilog Debugging Framework. *arXiv preprint arXiv:2606.23759*, 2026. URL <https://arxiv.org/abs/2606.23759>.
- [233] Yunan Wang, Tingyu Guo, Zilong Huang, and Yuan Yuan. Revisiting Evolutionary Program Repair via Code Language Model. *arXiv preprint arXiv:2408.10486*, 2024. URL <https://arxiv.org/abs/2408.10486>.
- [234] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. Copiloting the Copilots: Fusing Large Language Models with Completion Engines for Automated Program Repair. *arXiv preprint arXiv:2309.00608*, 2023. URL <https://arxiv.org/abs/2309.00608>.
- [235] Martin Weiss, Jesko Hecking-Harbusch, Jochen Quante, and Matthias Woehrle. Feedback Loops and Code Perturbations in LLM-based Software Engineering: A Case Study on a C-to-Rust Translation System. *arXiv preprint arXiv:2512.02567*, 2025. URL <https://arxiv.org/abs/2512.02567>.
- [236] Kyle Wong, Alfonso Amayuelas, Liangming Pan, and William Yang Wang. Investigating the Transferability of Code Repair for Low-Resource Programming Languages. *arXiv preprint arXiv:2406.14867*, 2024. URL <https://arxiv.org/abs/2406.14867>.
- [237] Linhao Wu, Yifei Pei, Zhen Yang, Kainan Li, Zhonghang Lu, Hao Tan, Xiran Lyu, Jia Li, Yizhou Chen, Pengyu Xue, Kunwu Zheng, and Dan Hao. DebugRepair: Enhancing LLM-Based Automated Program Repair via Self-Directed Debugging. *arXiv preprint arXiv:2604.19305*, 2026. URL <https://arxiv.org/abs/2604.19305>.
- [238] Valentina Wu, Alexandra Mendes, and Alexandre Abreu. Specification-Guided Repair of Arithmetic Errors in Dafny Programs using LLMs. *arXiv preprint arXiv:2507.03659*, 2025. URL <https://arxiv.org/abs/2507.03659>.
- [239] Yonghao Wu, Zheng Li, Jie M. Zhang, and Yong Liu. ConDefects: A New Dataset to Address the Data Leakage Concern for LLM-based Fault Localization and Program Repair. *arXiv preprint arXiv:2310.16253*, 2023. URL <https://arxiv.org/abs/2310.16253>.
- [240] Yonghao Wu, Zheng Li, Jie M. Zhang, Mike Papadakis, Mark Harman, and Yong Liu. Large Language Models in Fault Localisation. *arXiv preprint arXiv:2308.15276*, 2023. URL <https://arxiv.org/abs/2308.15276>.
- [241] Chunqiu Steven Xia and Lingming Zhang. Keep the Conversation Going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. *arXiv preprint arXiv:2304.00385*, 2023. URL <https://arxiv.org/abs/2304.00385>.
- [242] Chunqiu Steven Xia, Yifeng Ding, and Lingming Zhang. Revisiting the Plastic Surgery Hypothesis via Large Language Models. *arXiv preprint arXiv:2303.10494*, 2023. URL <https://arxiv.org/abs/2303.10494>.
- [243] Jiahong Xiang, Xiaoyang Xu, Fanchu Kong, Mingyuan Wu, Zizheng Zhang, Haotian Zhang, and Yuqun Zhang. How Far Can We Go with Practical Function-Level Program Repair? *arXiv preprint arXiv:2404.12833*, 2024. URL <https://arxiv.org/abs/2404.12833>.
- [244] YingJian Xiao, RongQun Hu, WeiWei Gong, HongWei Li, and AnQuan Jie. Large Language Models for Fault Localization: An Empirical Study. *arXiv preprint arXiv:2510.20521*, 2025. URL <https://arxiv.org/abs/2510.20521>.
- [245] Bang Xie, Senjian Zhang, Zhiyuan Peng, Wei Chen, Chenhao Ying, and Yuan Luo. ArkEval: Benchmarking and Evaluating Automated CodeRepair for ArkTS. *arXiv preprint arXiv:2602.08866*, 2026. URL <https://arxiv.org/abs/2602.08866>.
- [246] Qi Xin, Haojun Wu, Steven P. Reiss, and Jifeng Xuan. Towards Practical and Useful Automated Program Repair for Debugging. *arXiv preprint arXiv:2407.08958*, 2024. URL <https://arxiv.org/abs/2407.08958>.
- [247] Huiyu Xu, Wenhui Zhang, Zhibo Wang, Feng Xiao, Rui Zheng, Yunhe Feng, Zhongjie Ba, and Kui Ren. RedAgent: Red Teaming Large Language Models with Context-aware Autonomous Language Agent. *arXiv preprint arXiv:2407.16667*, 2024. URL <https://arxiv.org/abs/2407.16667>.
- [248] Jingxuan Xu, Ken Deng, Weihao Li, Songwei Yu, Huaixi Tang, Haoyang Huang, Zhiyi Lai, Zizheng Zhan, Yanan Wu, Chenchen Zhang, Kepeng Lei, Yifan Yao, Xinpeng Lei, Wenqiang Zhu, Zongxian Feng, Han Li, Junqi Xiong, Dailin Li, Zuchen Gao, Kun Wu, et al. SWE-Compass: Towards Unified Evaluation of Agentic Coding Abilities for Large Language Models. *arXiv preprint arXiv:2511.05459*, 2025. URL <https://arxiv.org/abs/2511.05459>.
- [249] Junjielong Xu, Ying Fu, Shin Hwei Tan, and Pinjia He. Aligning the Objective of LLM-based Program Repair. *arXiv preprint arXiv:2404.08877*, 2024. URL <https://arxiv.org/abs/2404.08877>.

- [250] Kangwei Xu, Grace Li Zhang, Xunzhao Yin, Cheng Zhuo, Ulf Schlichtmann, and Bing Li. Automated C/C++ Program Repair for High-Level Synthesis via Large Language Models. *arXiv preprint arXiv:2407.03889*, 2024. URL <https://arxiv.org/abs/2407.03889>.
- [251] Qinghua Xu, Guancheng Wang, Lionel Briand, Zhaoqiang Guo, and Kui Liu. LLM-based Mockless Unit Test Generation for Java. *arXiv preprint arXiv:2605.26851*, 2026. URL <https://arxiv.org/abs/2605.26851>.
- [252] WeiZhe Xu, Mengyu Liu, and Fanxin Kong. Enhancing LLM-Based Test Generation by Eliminating Covered Code. *arXiv preprint arXiv:2602.21997*, 2026. URL <https://arxiv.org/abs/2602.21997>.
- [253] Wendong Xu, Jing Xiong, Chenyang Zhao, Qiujiang Chen, Haoran Wang, Hui Shen, Zhongwei Wan, Jianbo Dai, Taiqiang Wu, He Xiao, Chaofan Tao, Z. Morley Mao, Ying Sheng, Zhijiang Guo, Hongxia Yang, Bei Yu, Lingpeng Kong, Quanquan Gu, and Ngai Wong. SwingArena: Competitive Programming Arena for Long-context GitHub Issue Solving. *arXiv preprint arXiv:2505.23932*, 2025. URL <https://arxiv.org/abs/2505.23932>.
- [254] Pengyu Xue, Linhao Wu, Zhen Yang, Zhongxing Yu, Zhi Jin, Ge Li, Yan Xiao, Shuo Liu, Xinyi Li, Hongyi Lin, and Jingwen Wu. Exploring and Lifting the Robustness of LLM-powered Automated Program Repair with Metamorphic Testing. *arXiv preprint arXiv:2410.07516*, 2024. URL <https://arxiv.org/abs/2410.07516>.
- [255] Aashish Yadavally, Hoan Nguyen, Laurent Callot, and Gauthier Guinet. Large Language Model Critics for Execution-Free Evaluation of Code Changes. *arXiv preprint arXiv:2501.16655*, 2025. URL <https://arxiv.org/abs/2501.16655>.
- [256] Katsuki Yamagishi, Norihiro Yoshida, Erina Makihara, and Katsuro Inoue. Automated Program Repair Based on REST API Specifications Using Large Language Models. *arXiv preprint arXiv:2510.25148*, 2025. URL <https://arxiv.org/abs/2510.25148>.
- [257] Aidan Z. H. Yang, Ruben Martins, Claire Le Goues, and Vincent J. Hellendoorn. Large Language Models for Test-Free Fault Localization. *arXiv preprint arXiv:2310.01726*, 2023. URL <https://arxiv.org/abs/2310.01726>.
- [258] Aidan Z. H. Yang, Sophia Kolak, Vincent J. Hellendoorn, Ruben Martins, and Claire Le Goues. Revisiting Unnaturalness for Automated Program Repair in the Era of Large Language Models. *arXiv preprint arXiv:2404.15236*, 2024. URL <https://arxiv.org/abs/2404.15236>.
- [259] Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawende F. Bissyande, and Shunfu Jin. CREF: An LLM-based Conversational Software Repair Framework for Programming Tutors. *arXiv preprint arXiv:2406.13972*, 2024. URL <https://arxiv.org/abs/2406.13972>.
- [260] Boyang Yang, Zijian Cai, Fengling Liu, Bach Le, Lingming Zhang, Tegawende F. Bissyande, Yang Liu, and Haoye Tian. A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications. *arXiv preprint arXiv:2506.23749*, 2025. URL <https://arxiv.org/abs/2506.23749>.
- [261] Boyang Yang, Jiadong Ren, Shunfu Jin, Yang Liu, Feng Liu, Bach Le, and Haoye Tian. Enhancing repository-level software repair via repository-aware knowledge graphs. *arXiv preprint arXiv:2503.21710*, 2025. URL <https://arxiv.org/abs/2503.21710>.
- [262] Boyang Yang, Luyao Ren, Xin Yin, Jiadong Ren, Haoye Tian, and Shunfu Jin. Input Reduction Enhanced LLM-based Program Repair. *arXiv preprint arXiv:2507.15251*, 2025. URL <https://arxiv.org/abs/2507.15251>.
- [263] Boyang Yang, Zijian Cai, Shunfu Jin, and Haoye Tian. PAFT: Preservation Aware Fine-Tuning for Minimal-Edit Program Repair. *arXiv preprint arXiv:2604.03113*, 2026. URL <https://arxiv.org/abs/2604.03113>.
- [264] Chen Yang, Junjie Chen, Bin Lin, Ziqi Wang, and Jianyi Zhou. Advancing Code Coverage: Incorporating Program Analysis with Large Language Models. *arXiv preprint arXiv:2404.04966*, 2024. URL <https://arxiv.org/abs/2404.04966>.
- [265] Chen Yang, Ziqi Wang, Yanjie Jiang, Lin Yang, Yuteng Zheng, Jianyi Zhou, and Junjie Chen. Reflective Unit Test Generation for Precise Type Error Detection with Large Language Models. *arXiv preprint arXiv:2507.02318*, 2025. URL <https://arxiv.org/abs/2507.02318>.
- [266] Chen Yang, Lin Yang, Ziqi Wang, Dong Wang, Jianyi Zhou, and Junjie Chen. Clarifying Semantics of In-Context Examples for Unit Test Generation. *arXiv preprint arXiv:2510.01994*, 2025. URL <https://arxiv.org/abs/2510.01994>.
- [267] Lekang Yang, Yuetong Liu, Yitong Zhang, and Jia Li. DiffuTester: Accelerating Unit Test Generation for Diffusion LLMs via Mining Structural Pattern. *arXiv preprint arXiv:2509.24975*, 2025. URL <https://arxiv.org/abs/2509.24975>.
- [268] Lin Yang, Chen Yang, Shutao Gao, Weijing Wang, Bo Wang, Qihao Zhu, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, and Junjie Chen. On the Evaluation of Large Language

- Models in Unit Test Generation. *arXiv preprint arXiv:2406.18181*, 2024. URL <https://arxiv.org/abs/2406.18181>.
- [269] Weiqing Yang, Hanbin Wang, Zhenghao Liu, Xinze Li, Yukun Yan, Shuo Wang, Yu Gu, Minghe Yu, Zhiyuan Liu, and Ge Yu. COAST: Enhancing the Code Debugging Ability of LLMs through Communicative Agent Based Data Synthesis. *arXiv preprint arXiv:2408.05006*, 2024. URL <https://arxiv.org/abs/2408.05006>.
- [270] Bingkun Yao, Ning Wang, Jie Zhou, Xi Wang, Hong Gao, Zhe Jiang, and Nan Guan. Location is Key: Leveraging Large Language Model for Functional Bug Localization in Verilog. *arXiv preprint arXiv:2409.15186*, 2024. URL <https://arxiv.org/abs/2409.15186>.
- [271] Bingkun Yao, Ning Wang, Xiangfeng Liu, Yuxin Du, Yuchen Hu, Hong Gao, Zhe Jiang, and Nan Guan. ARSP: Automated Repair of Verilog Designs via Semantic Partitioning. *arXiv preprint arXiv:2508.16517*, 2025. URL <https://arxiv.org/abs/2508.16517>.
- [272] Yinsheng Yao, Hongxiang Zhang, Weixi Tong, and Tianyi Zhang. FLARE: Fine-Grained Diagnostic Feedback for LLM Code Refinement. *arXiv preprint arXiv:2606.03852*, 2026. URL <https://arxiv.org/abs/2606.03852>.
- [273] Zhenlei Ye, Xiaobing Sun, Sicong Cao, Lili Bo, and Bin Li. Well Begun is Half Done: Location-Aware and Trace-Guided Iterative Automated Vulnerability Repair. *arXiv preprint arXiv:2512.20203*, 2025. URL <https://arxiv.org/abs/2512.20203>.
- [274] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. ThinkRepair: Self-Directed Automated Program Repair. *arXiv preprint arXiv:2407.20898*, 2024. URL <https://arxiv.org/abs/2407.20898>.
- [275] Chihiro Yoshida, Yuta Ishimoto, Olivier Nourry, Masanari Kondo, Makoto Matsushita, Yasutaka Kamei, and Yoshiki Higo. Leveraging Mutation Analysis for LLM-based Repair of Quantum Programs. *arXiv preprint arXiv:2601.12273*, 2026. URL <https://arxiv.org/abs/2601.12273>.
- [276] Xiao Yu, Haoxuan Chen, Feifei Niu, Xing Hu, Jacky Wai Keung, and Xin Xia. Towards Understanding Bugs in Distributed Training and Inference Frameworks for Large Language Models. *arXiv preprint arXiv:2506.10426*, 2025. URL <https://arxiv.org/abs/2506.10426>.
- [277] Zhongming Yu, Hejia Zhang, Yujie Zhao, Hanxian Huang, Matrix Yao, Ke Ding, and Jishen Zhao. OrcaLoca: An LLM Agent Framework for Software Issue Localization. *arXiv preprint arXiv:2502.00350*, 2025. URL <https://arxiv.org/abs/2502.00350>.
- [278] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. No More Manual Tests? Evaluating and Improving ChatGPT for Unit Test Generation. *arXiv preprint arXiv:2305.04207*, 2023. URL <https://arxiv.org/abs/2305.04207>.
- [279] Jiayi Zhang, Kai Huang, Jian Zhang, Yang Liu, and Chunyang Chen. Repair Ingredients Are All You Need: Improving Large Language Model-Based Program Repair via Repair Ingredients Search. *arXiv preprint arXiv:2506.23100*, 2025. URL <https://arxiv.org/abs/2506.23100>.
- [280] Jingyao Zhang, Tianlin Li, Xiaoyu Zhang, Qiang Hu, and Bin Shi. Exploring the Power of Diffusion Large Language Models for Software Engineering: An Empirical Investigation. *arXiv preprint arXiv:2510.04605*, 2025. URL <https://arxiv.org/abs/2510.04605>.
- [281] Linhao Zhang, Daoguang Zan, Quanshun Yang, Zhirong Huang, Dong Chen, Bo Shen, Tianyu Liu, Yongshun Gong, Pengjie Huang, Xudong Lu, Guangtai Liang, Lizhen Cui, and Qianxiang Wang. CodeV: Issue Resolving with Visual Data. *arXiv preprint arXiv:2412.17315*, 2024. URL <https://arxiv.org/abs/2412.17315>.
- [282] Quanjun Zhang, Tongke Zhang, Juan Zhai, Chunrong Fang, Bowen Yu, Weisong Sun, and Zhenyu Chen. A Critical Review of Large Language Model on Software Engineering: An Example from ChatGPT and Automated Program Repair. *arXiv preprint arXiv:2310.08879*, 2023. URL <https://arxiv.org/abs/2310.08879>.
- [283] Quanjun Zhang, Chunrong Fang, Ye Shang, Tongke Zhang, Shengcheng Yu, and Zhenyu Chen. No Man is an Island: Towards Fully Automatic Programming by Code Search, Code Generation and Program Repair. *arXiv preprint arXiv:2409.03267*, 2024. URL <https://arxiv.org/abs/2409.03267>.
- [284] Quanjun Zhang, Chunrong Fang, Yang Xie, YuXiang Ma, Weisong Sun, Yun Yang, and Zhenyu Chen. A Systematic Literature Review on Large Language Models for Automated Program Repair. *arXiv preprint arXiv:2405.01466*, 2024. URL <https://arxiv.org/abs/2405.01466>.
- [285] Quanjun Zhang, Chunrong Fang, Siqi Gu, Ye Shang, Zhenyu Chen, and Liang Xiao. Large Language Models for Unit Testing: A Systematic Literature Review. *arXiv preprint arXiv:2506.15227*, 2025. URL <https://arxiv.org/abs/2506.15227>.
- [286] Quanjun Zhang, Chengyu Gao, Yu Han, Ye Shang, Chunrong Fang, Zhenyu Chen, and Liang Xiao. SGAgent: Suggestion-Guided LLM-Based Multi-Agent Framework for Repository-Level Software

- Repair. *arXiv preprint arXiv:2602.23647*, 2026. URL <https://arxiv.org/abs/2602.23647>.
- [287] Yukuan Zhang, Mengxin Zheng, and Qian Lou. MPC-Patch-Bench: Security-Aware LLM Code Patch for Multi-Party Computation. *arXiv preprint arXiv:2606.11416*, 2026. URL <https://arxiv.org/abs/2606.11416>.
- [288] Yuntong Zhang, Jiawei Wang, Dominic Berzin, Martin Mirchev, Dongge Liu, Abhishek Arya, Oliver Chang, and Abhik Roychoudhury. Fixing Security Vulnerabilities with AI in OSS-Fuzz. *arXiv preprint arXiv:2411.03346*, 2024. URL <https://arxiv.org/abs/2411.03346>.
- [289] Yuwei Zhang, Zhi Jin, Ying Xing, Ge Li, Fang Liu, Jiaxin Zhu, Wensheng Dou, and Jun Wei. PATCH: Empowering Large Language Model with Programmer-Intent Guidance and Collaborative-Behavior Simulation for Automatic Bug Fixing. *arXiv preprint arXiv:2501.16149*, 2025. URL <https://arxiv.org/abs/2501.16149>.
- [290] Yuwei Zhang, Qingyuan Lu, Kai Liu, Wensheng Dou, Jiaxin Zhu, Li Qian, Chunxi Zhang, Zheng Lin, and Jun Wei. CITYWALK: Enhancing LLM-Based C++ Unit Test Generation via Project-Dependency Awareness and Language-Specific Knowledge. *arXiv preprint arXiv:2501.16155*, 2025. URL <https://arxiv.org/abs/2501.16155>.
- [291] Zhe Zhang, Xingyu Liu, Yuanzhang Lin, Xiang Gao, Hailong Sun, and Yuan Yuan. LLM-based Unit Test Generation via Property Retrieval. *arXiv preprint arXiv:2410.13542*, 2024. URL <https://arxiv.org/abs/2410.13542>.
- [292] Yicong Zhao, Shisong Chen, Jiacheng Zhang, and Zhixu Li. ReCode: Improving LLM-based Code Repair with Fine-Grained Retrieval-Augmented Generation. *arXiv preprint arXiv:2509.02330*, 2025. URL <https://arxiv.org/abs/2509.02330>.
- [293] Zixiao Zhao, Amirreza Esmaeili, and Fatemeh Fard. Bias in the Loop: Auditing LLM-as-a-Judge for Software Engineering. *arXiv preprint arXiv:2604.16790*, 2026. URL <https://arxiv.org/abs/2604.16790>.
- [294] Han Zheng, Ilia Shumailov, Tianqi Fan, Aiden Hall, and Mathias Payer. Fixing 7,400 Bugs for 1\$: Cheap Crash-Site Program Repair. *arXiv preprint arXiv:2505.13103*, 2025. URL <https://arxiv.org/abs/2505.13103>.
- [295] Qixing Zhou, Jiacheng Zhang, Haiyang Wang, Rui Hao, Jiahe Wang, Minghao Han, Yuxue Yang, Shuzhe Wu, Feiyang Pan, Lue Fan, Dandan Tu, and Zhaoxiang Zhang. FeatureBench: Benchmarking Agentic Coding for Complex Feature Development. *arXiv preprint arXiv:2602.10975*, 2026. URL <https://arxiv.org/abs/2602.10975>.
- [296] Shuaiyu Zhou, Zhengran Zeng, Xiaoling Zhou, Rui Xie, Shikun Zhang, and Wei Ye. Seed&Steer: Guiding Large Language Models with Compilable Prefix and Branch Signals for Unit Test Generation. *arXiv preprint arXiv:2507.17271*, 2025. URL <https://arxiv.org/abs/2507.17271>.
- [297] Xin Zhou, Bowen Xu, Kisub Kim, DongGyun Han, Thanh Le-Cong, Junda He, Bach Le, and David Lo. PatchZero: Zero-Shot Automatic Patch Correctness Assessment. *arXiv preprint arXiv:2303.00202*, 2023. URL <https://arxiv.org/abs/2303.00202>.
- [298] Xin Zhou, Kisub Kim, Ting Zhang, Martin Weyssow, Luis F. Gomes, Guang Yang, Kui Liu, Xin Xia, and David Lo. An LLM-as-Judge Metric for Bridging the Gap with Human Evaluation in SE Tasks. *arXiv preprint arXiv:2505.20854*, 2025. URL <https://arxiv.org/abs/2505.20854>.
- [299] Xin Zhou, Martin Weyssow, Ratnadira Widyasari, Ting Zhang, Junda He, Yunbo Lyu, Jianming Chang, Beiqi Zhang, Dan Huang, and David Lo. LessLeak-Bench: A First Investigation of Data Leakage in LLMs Across 83 Software Engineering Benchmarks. *arXiv preprint arXiv:2502.06215*, 2025. URL <https://arxiv.org/abs/2502.06215>.
- [300] Taohong Zhu, Lucas C. Cordeiro, Mustafa A. Mustafa, and Youcheng Sun. Specification Vibing for Automated Program Repair. *arXiv preprint arXiv:2602.08263*, 2026. URL <https://arxiv.org/abs/2602.08263>.